

Building a Nepali Summarization Transformer from Scratch

Complete Documentation

Nepali Summarizer project

Contents

I	Data Preprocessing	3
1	Byte-Pair Encoding (BPE) Tokenization for Devanagari Text	4
	Architecture Overview	4
	Phase 1: Training the Tokenizer (Building the Dictionary)	4
	Phase 2: Tokenization (Encoding New Input)	6
	Phase 3: Decoding (Translating Back to Human Text)	7
II	The Transformer Encoder	8
2	Transformer Input Embedding Layer	9
	1. Matrix Architecture & Dimensions	9
	2. Initialization & Theoretical Foundations	10
	3. The Step-by-Step Pipeline	10
3	The Complete Positional Encoding & Input Preparation Pipeline	12
	1. Mathematical Generation of Positional Encodings	12
	2. Element-Wise Addition for Single Sequences	13
	3. Batching Multiple Sequences (3D Tensors)	14
	4. Padding Variable-Length Sequences	14
	5. Applying the Attention Mask	15
4	Multi-Head Self-Attention	17
	Phase 1: Data Ingestion (The Starting State)	17
	Phase 2: Single-Head Attention Pipeline	17
	Phase 3: Multi-Head Attention (MHA) Pipeline	19
5	Add & Norm (after Multi-Head Attention)	22
	Phase 1: The "Add" Component (Residual Connections)	22
	Phase 2: The "Norm" Component (Layer Normalization)	23
	Worked Example ($d_{model} = 4$, one token)	24
	Phase 3: The Complete Add & Norm Pipeline	24
6	Position-wise Feed-Forward Network (FFN)	26
	1. The Core Architecture	26
	2. The Expansion and Compression Strategy	26
	3. Worked Example (one token, $d_{model} = 2$, $d_{ff} = 4$)	27
	4. The Final Tensor Flow	28
7	Second Add & Norm (after the Feed-Forward Network)	29
	1. The "Add" Component (Second Residual Connection)	29

2. The "Norm" Component (Second Layer Normalization)	30
3. Worked Example (continuing the FFN example, $d_{model} = 2$)	30
4. Top-to-Bottom Flow of the Second Add & Norm Layer	30
Summary of the Entire Encoder Block Cycle	31
8 Linearization (LM Head) and Softmax	32
1. The Linearization and Softmax Head	32
2. What Is the Target? (Masked Language Modelling)	33
3. Initial Guesses vs. Actual Training (Updating Embeddings)	35
9 End-to-End Encoder Pipeline	37
Complete End-to-End Transformer Encoder Pipeline	37
Stage 1: Input Embedding & Positional Encoding	38
Stage 2: The Stacked Encoder Blocks (1 to N)	38
Stage 3: Sequential Block Passing	40
Stage 4: Language Modeling Head	40
Stage 5: Forward Pass vs. Training Updates (The Distinction)	40
Project-Specific Training and Target Data Preparation Strategy	41
One Training Step in This Project	43
Total Parameter Count	44
10 Backpropagation: How Every Weight and Bias Gets Updated	45
0. The Big Picture	45
1. Three Rules That Cover Everything	46
2. Step 1 — Loss and Softmax: $d\mathbf{Z}$	47
3. Step 2 — LM Head: W_{LM}, b_{LM}	48
4. Step 3 — Second Add & Norm: γ_2, β_2	48
5. Step 4 — Feed-Forward Network: W_2, b_2, W_1, b_1	49
6. Step 5 — First Add & Norm: γ_1, β_1	50
7. Step 6 — Multi-Head Attention: W_O, W_Q, W_K, W_V	50
8. Step 7 — Through the Stack of Blocks	51
9. Step 8 — Positional Encoding and Input Embedding: E	52
10. Updating the Parameters	52
11. Summary: Every Gradient in One Table	53
12. How We Know the Formulas Are Right: Gradient Checking	54

Part I

Data Preprocessing

Chapter 1

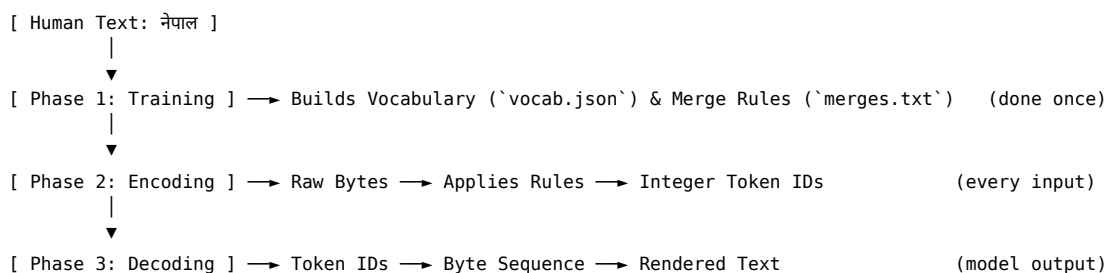
Byte-Pair Encoding (BPE) Tokenization for Devanagari Text

This document explains how modern Large Language Model (LLM) tokenizers handle multi-byte scripts like Devanagari using **Byte-Level Byte-Pair Encoding (BPE)**, with real numbers from this project's tokenizer.

In one sentence: text is turned into bytes, and the tokenizer has learned which byte sequences occur together often enough to deserve their own token ID.

Code: training in `src/data_preprocessing/bpe_tokenization.py` (python `-m scripts.train_nepali_bpe`); output files in `data/nepali_vocab_output/` (`vocab.json`, `merges.txt`); used by `src/data_preprocessing/batching.py`.

Architecture Overview



Phase 1: Training the Tokenizer (Building the Dictionary)

1. Base Initialization (Special Tokens + The 256 Bytes)

The tokenizer starts with a fixed, unbreakable foundation of exactly **256 base tokens**. These represent every possible raw 8-bit computer byte:

Range: 00000000_2 to 11111111_2 (Decimal 0 to 255)

Because every text is made of bytes, **any** input can be encoded — there is never an “unknown character”.

In this project, 5 **special tokens** are added *before* the bytes, so they take the first IDs:

ID	Token	Purpose
0	<s>	beginning of sequence
1	<pad>	padding (see encoders/2-positional_encoding.md)
2	</s>	end of sequence
3	<unk>	unknown (rarely needed with byte-level BPE)
4	<mask>	hidden token for masked language modelling
5 – 260	the 256 bytes	base alphabet
261 – 6301	learned merges	built during training

2. Raw Byte Breakdown

The entire Devanagari training corpus is broken down into a stream of single bytes.

UTF-8 is a variable-width encoding: English letters take 1 byte, but **every Devanagari character takes 3 bytes**. For example:

Character 'न' (U+0928) $\rightarrow$ [224, 164, 168]

Almost every Devanagari character starts with byte 224 followed by 164 or 165 — which is why those pairs get merged first (see below).

3. Frequency Counting

The algorithm scans the byte sequences across the dataset and counts how often every adjacent pair of tokens occurs.

The tokenizer first splits text at spaces (a “pre-tokenization” step), so merges never cross word boundaries. A space is kept attached to the start of the next word.

4. Statistical Merging

The single most frequent pair in the text is merged to create a brand-new token, assigned the next available ID.

- **Real example from this project:** the first merge learned was byte 224 + byte 164 — the shared prefix of most Devanagari characters. It became **ID 261** (the first ID after the 5 special tokens and 256 bytes).

5. Dictionary Mapping

The tokenizer records each new token in two files:

- **Vocabulary File (vocab.json):** token $\rightarrow$ ID, e.g. the token for bytes [224, 164] $\rightarrow$ 261.
- **Rulebook (merges.txt):** the ordered list of merges, one per line (first line = first merge).

Why do the files look like à¸” instead of Nepali? Byte-level BPE stores each byte as a printable stand-in character so the files are valid text: byte 224 $\rightarrow$ à, 164 $\rightarrow$ ¸, 165 $\rightarrow$ ¨, a space $\rightarrow$ Ġ, and so on. So à¸” in vocab.json is simply bytes

[224, 164, 168] = न. The first line of our merges.txt is न, i.e. "merge byte 224 with byte 164".

6. Iteration and Expansion

The tokenizer replaces every occurrence of the merged pair with the new single token, recounts adjacent pairs, and repeats:

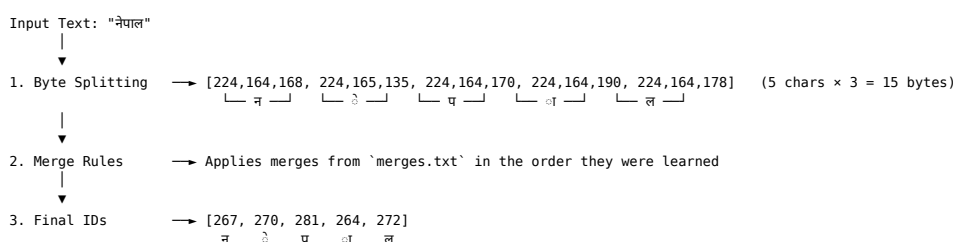
1. Merges [224, 164] (ID 261) with [168] to create the full character न (ID 267 in our vocabulary).
2. Continues merging characters into common syllables and word pieces.
3. Merges pieces into frequent words.

This loop repeats until the vocabulary reaches the target size, or until no pair is frequent enough (our training requires a pair to appear at least twice, min_frequency=2).

This project: trained on our ~132,000 cleaned Nepali news articles, the loop reaches the full target of 30,000 **tokens** ($V = 30000$: 5 special tokens + 256 bytes + 29,739 merges). Large models use 32,000–50,000 or more.

Phase 2: Tokenization (Encoding New Input)

When processing a new piece of text, the trained tokenizer performs three steps. Real output for "नेपाल" (5 Unicode characters: न + े + प + ा + ल):



1. **Byte Splitting:** The input text is split into its raw UTF-8 bytes.
2. **Applying Merge Rules:** The tokenizer executes learned merges in the exact order they were created during training.
3. **Final ID Output:** Merged chunks are replaced with their integer IDs from vocab.json. These IDs are passed into the model's **Embedding Layer** (encoders/1-input_embedding.md).

Even with 30,000 tokens, "नेपाल" becomes one token per character. Runs of consonants do become single tokens. For example, in "नेपाल सरकार" the piece "सरक" (space + स + र + क) is a single token (ID 413):

Text	Token IDs	Count
न	[267]	1
नेपाल	[267, 270, 281, 264, 272]	5
नेपाल सरकार	[267, 270, 281, 264, 272, 413, 264, 266]	8 (31 bytes)
काठमाडौं	[268, 264, 502, 264, 316, 442]	6 (24 bytes)

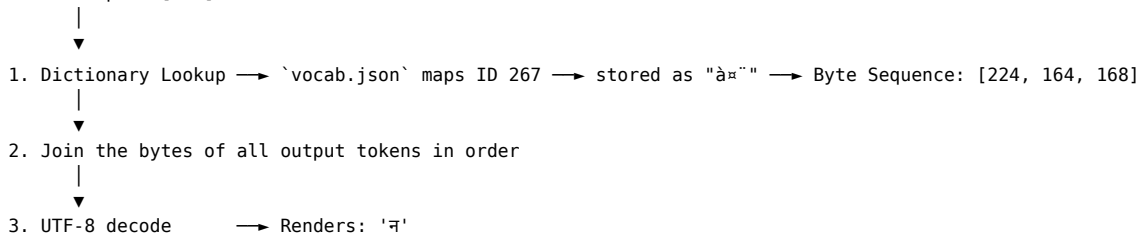
Why a bigger vocabulary does not merge whole words here: before BPE runs, ByteLevelBPETokenizer pre-splits text with the GPT-2 regex, which groups

letters with `\p{L}+`. Devanagari vowel signs (matras such as `े` and `ा`) and the anusvara `ं` are Unicode *marks* (categories Mn/Mc), not letters. So “नेपाल” is cut into न | े | प | ा | ल before any merging, and BPE can never merge across a matra. That is why the learned tokens are consonant runs like “स्रक्”, and why our corpus averages only about 1.5 characters per token. A pre-tokenizer that keeps letters and marks together (e.g. splitting on `\s?[\p{L}\p{M}]+`) would let common words like नेपाल become single tokens and make sequences shorter.

Phase 3: Decoding (Translating Back to Human Text)

When the model outputs token IDs, the process runs in reverse to render readable text:

Model Output: [267]



1. **Dictionary Lookup:** The tokenizer takes an integer ID (e.g., 267) and finds its token in `vocab.json`.
2. **Byte Retrieval:** It converts the stored stand-in characters back to the raw bytes ([224, 164, 168]) and joins the bytes of all tokens.
3. **UTF-8 Decoding:** The byte sequence is decoded as UTF-8 and shown as the glyph न.

Important: a single token can be *part* of a character (e.g. ID 261 = bytes [224, 164] only). Always decode the **whole** list of IDs together (`tokenizer.decode(ids)`), not one token at a time — otherwise you may get broken characters or à¤-style strings.

Part II

The Transformer Encoder

Chapter 2

Transformer Input Embedding Layer

[Input Token IDs] $\rightarrow$ [Matrix Lookup (Row Index)] $\rightarrow$ [Vector Scaling ($\times \sqrt{d_{\text{model}}}$)] $\rightarrow$ [Scaled Input Embeddings]
(e.g., [258, 890]) $E[258, :], E[890, :]$ Multiply by $\sqrt{d_{\text{model}}}$ Ready for Positional Encoding

In one sentence: every token ID is just a row number; the embedding layer fetches that row from a big table of numbers and multiplies it by $\sqrt{d_{\text{model}}}$.

	Shape	This project
Input (token IDs)	(B, T)	$(4, 512)$ integers
Embedding matrix E	(V, d_{model})	$(30000, 512)$
Output	(B, T, d_{model})	$(4, 512, 512)$

Code: `src/models/layers/input_embedding.py` $\rightarrow$ `InputEmbedding`

1. Matrix Architecture & Dimensions

The embedding layer consists of a lookup table represented by the 2D matrix E with dimensions $V \times d_{\text{model}}$:

- **V (Vocabulary Size):** The total number of unique tokens created during the Byte-Pair Encoding (BPE) process. Large models use e.g. 50,000; **our Nepali BPE vocabulary has $V = 30,000$.**
- **d_{model} (Embedding Dimension):** The width of the Transformer's hidden vector space (e.g., 512 or 4096). **We use $d_{\text{model}} = 512$.**

Row r of the matrix is the vector for token ID r . Rows are numbered from 0 to $V - 1$ (matching token IDs), columns from 1 to d_{model} :

$$\text{Embedding Matrix } E = \begin{bmatrix} e_{0,1} & e_{0,2} & \dots & e_{0,d_{\text{model}}} \\ e_{1,1} & e_{1,2} & \dots & e_{1,d_{\text{model}}} \\ \vdots & \vdots & \ddots & \vdots \\ e_{V-1,1} & e_{V-1,2} & \dots & e_{V-1,d_{\text{model}}} \end{bmatrix}$$

Size in this project: $30000 \times 512 = 15,360,000$ learnable numbers.

2. Initialization & Theoretical Foundations

When the Transformer is instantiated, matrix E is populated with small random numbers drawn from a normal distribution with **mean 0 and standard deviation 0.02**:

$$E_{i,j} \sim \mathcal{N}(\mu = 0, \sigma = 0.02)$$

Notation note: here the second number is the *standard deviation* σ (not the variance $\sigma^2 = 0.0004$). In NumPy: `np.random.normal(0, 0.02, size=(V, d_model))`. GPT and BERT use a *truncated* normal (values beyond $\pm 2\sigma$ are re-drawn). Our code uses a plain normal; with $\sigma = 0.02$ the difference is negligible.

Why $\sigma = 0.02$ is Used

- **Preventing Exploding Gradients:** Deep neural networks multiply these weights across dozens of layers. If starting values are drawn from $\mathcal{N}(0, 1)$, numbers grow exponentially during matrix multiplications, leading to exploding gradients and training instability.
 - **Avoiding Softmax Saturation:** Large initial values push output predictions to extreme probabilities (approaching 0 or 1). This saturates the softmax activation function, dropping loss gradients to near zero and freezing the learning process. With $\sigma = 0.02$, about 99.7% of values fall within $\pm 3\sigma = [-0.06, 0.06]$, which keeps all predictions neutral and highly responsive to updates.
-

3. The Step-by-Step Pipeline

Step 1: Matrix Lookup

For an input sequence of T token IDs $\mathbf{x} = [x_1, x_2, \dots, x_T]$ where each $x_i \in [0, V - 1]$, the layer extracts the corresponding row vectors from E :

$$\mathbf{v}_i = E[x_i, :] \in \mathbb{R}^{d_{\text{model}}}$$

This yields an unscaled sequence matrix $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{T \times d_{\text{model}}}$. No multiplication happens here — it is pure indexing (`E[token_ids]` in NumPy), which also works for a whole batch: $(B, T) \rightarrow (B, T, d_{\text{model}})$.

Step 2: Vector Scaling by $\sqrt{d_{\text{model}}}$

Each extracted vector is multiplied by the square root of the embedding dimension:

$$\mathbf{e}_i = \mathbf{v}_i \cdot \sqrt{d_{\text{model}}}$$

Why Scaling is Mandatory: The next component in the Transformer pipeline is adding Positional Encodings generated via sine and cosine functions, which strictly output values in the range $[-1, 1]$.

Because raw initial embedding values are tiny (around ± 0.02), adding positional encodings directly would completely drown out the token's content. For $d_{\text{model}} = 512$, $\sqrt{512} \approx 22.63$, so a typical value of 0.02 becomes $0.02 \times 22.63 \approx 0.45$ — now on the same scale as the positional signal. Positional information can then shift word order without washing out the word's base representation.

Worked Example (tiny numbers)

Use a toy vocabulary of $V = 6$ tokens and $d_{\text{model}} = 4$, so the scale factor is $\sqrt{4} = 2$.

$$E = \begin{bmatrix} 0.01 & -0.02 & 0.03 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 \\ 0.02 & 0.01 & -0.01 & 0.03 \\ -0.03 & 0.02 & 0.01 & -0.01 \\ 0.01 & 0.01 & 0.02 & -0.02 \\ 0.00 & -0.01 & 0.02 & 0.01 \end{bmatrix} \begin{matrix} \leftarrow \text{ID 0} \\ \leftarrow \text{ID 1} \\ \leftarrow \text{ID 2} \\ \leftarrow \text{ID 3} \\ \leftarrow \text{ID 4} \\ \leftarrow \text{ID 5} \end{matrix}$$

Input sentence with token IDs [2, 5, 3] ($T = 3$):

Token ID	Step 1: row $E[x_i]$	Step 2: $\times 2$
2	[0.02, 0.01, -0.01, 0.03]	[0.04, 0.02, -0.02, 0.06]
5	[0.00, -0.01, 0.02, 0.01]	[0.00, -0.02, 0.04, 0.02]
3	[-0.03, 0.02, 0.01, -0.01]	[-0.06, 0.04, 0.02, -0.02]

Output shape: $(T, d_{\text{model}}) = (3, 4)$; with a batch dimension, $(1, 3, 4)$. This matrix goes straight into the Positional Encoding step (the same numbers are continued in 2-positional_encoding.md).

Step 3: Acquiring Meaning via Backpropagation

At step zero, these vectors are merely arbitrary coordinates. Real semantic structure is carved out dynamically during training:

1. **Forward Pass:** The scaled tensor $\mathbf{X}_{\text{scaled}} \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ (plus positional encodings) enters the self-attention blocks.
2. **Loss Calculation:** The model predicts target tokens and computes loss $\mathcal{L}$.
3. **Gradient Update:** Backpropagation calculates partial derivatives $\frac{\partial \mathcal{L}}{\partial E}$, and an optimizer (we use Adam) updates the matrix values. In plain gradient descent this would be:

$$E[x_i, :] \leftarrow E[x_i, :] - \eta \cdot \frac{\partial \mathcal{L}}{\partial E[x_i, :]}$$

Only the rows of tokens that appeared in the batch get a non-zero gradient. The exact formula (including the $\sqrt{d_{\text{model}}}$ factor and repeated tokens) is derived in 9-backpropagation.md.

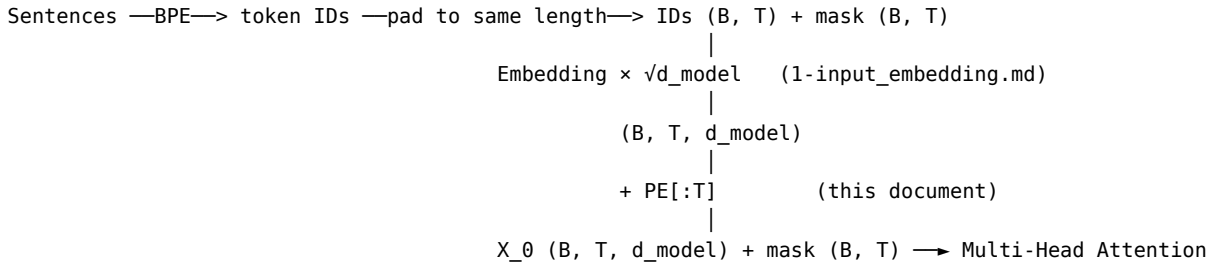
Over many iterations, tokens appearing in similar contexts receive similar gradient updates. Their vectors align in d_{model} -dimensional space, transforming random numbers into a structured semantic map.

Chapter 3

The Complete Positional Encoding & Input Preparation Pipeline

Because the Transformer processes all tokens in a sequence simultaneously, its underlying self-attention mechanism is **permutation-invariant**. It has no inherent concept of word order. To prevent the model from treating sentences as an unordered “bag of words,” **Positional Encodings** mathematically inject sequence order directly into the token representations.

The pipeline below details the complete journey from raw text to the final 3D batched tensor passed into the Multi-Head Attention layer.



Order matters: padding and the mask are created on the **token IDs** (before embedding). The [PAD] IDs then get embedded and position-encoded like any other token; the mask is what makes attention ignore them.

Code: `src/models/layers/positional_encoding.py` → `PositionalEncoding`;
padding and mask in `src/data_preprocessing/batching.py` → `CreateTrainingBatch.create_input_tensor`.

1. Mathematical Generation of Positional Encodings

The Transformer generates a static (not learned) Positional Encoding (PE) matrix of shape $(T_{\max}, d_{\text{model}})$ — in this project (512, 512) — using sine and cosine functions. For any token at sequence position pos (starting at 0) and dimension-pair index $i \in \left[0, \frac{d_{\text{model}}}{2} - 1\right]$:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

In words: dimensions come in pairs $(2i, 2i + 1)$. Both dimensions of a pair use the **same frequency** $\frac{1}{10000^{2i/d_{\text{model}}}}$; the even one takes the sine, the odd one takes the cosine.

- **Frequency Decay:** Lower dimensions (small i) oscillate rapidly to capture local, immediate word order. Higher dimensions (large i) change very slowly and track coarse, global position across long sequences.
- **Linear Relative Shift:** Because of trigonometric angle-addition identities, the encoding at position $pos + k$ is a fixed rotation of the encoding at position pos . This lets attention heads compute relative distances between tokens (e.g., "*three words to my left*"), regardless of absolute position.

Worked Example: PE for $d_{\text{model}} = 4$

With $d_{\text{model}} = 4$ there are two pairs:

- Pair $i = 0$ (dims 0, 1): frequency = $1/10000^{0/4} = 1 \rightarrow \text{angle} = pos$
- Pair $i = 1$ (dims 2, 3): frequency = $1/10000^{2/4} = 1/100 \rightarrow \text{angle} = pos/100$

pos	dim 0: $\sin(pos)$	dim 1: $\cos(pos)$	dim 2: $\sin(pos/100)$	dim 3: $\cos(pos/100)$
0	0.0000	1.0000	0.0000	1.0000
1	0.8415	0.5403	0.0100	1.0000
2	0.9093	-0.4161	0.0200	0.9998
3	0.1411	-0.9900	0.0300	0.9996

Dims 0–1 change a lot from one position to the next (fast "clock hand"); dims 2–3 barely move (slow "clock hand").

2. Element-Wise Addition for Single Sequences

For a single sentence like "**The cat ate fish**" (4 tokens), the model first retrieves the scaled semantic vectors from the Embedding Layer, resulting in a matrix of shape $4 \times d_{\text{model}}$ (e.g., 4×512).

It then takes the first 4 rows from the pre-computed PE matrix, which also has a shape of 4×512 . Instead of concatenating them, the model performs direct **element-wise addition**:

- **Row 1:** $\text{Embed}(\text{"The"}) + PE(0)$
- **Row 2:** $\text{Embed}(\text{"cat"}) + PE(1)$
- **Row 3:** $\text{Embed}(\text{"ate"}) + PE(2)$
- **Row 4:** $\text{Embed}(\text{"fish"}) + PE(3)$

Because 512 is a high-dimensional space, the semantic representation absorbs this positional signal without destroying its core identity. The output remains a 4×512 matrix containing both the **"what"** (semantics) and the **"where"** (position) of every token.

Worked Example (continuing from 1-input_embedding.md)

Scaled embeddings for token IDs [2, 5, 3] plus the first 3 rows of the PE table above:

pos	Scaled embedding	+ $PE(pos)$	= $\mathbf{X}_0$ row
0	[0.04, 0.02, -0.02, 0.06]	[0, 1, 0, 1]	[0.04, 1.02, -0.02, 1.06]
1	[0.00, -0.02, 0.04, 0.02]	[0.8415, 0.5403, 0.01, 1.0]	[0.8415, 0.5203, 0.05, 1.02]
2	[-0.06, 0.04, 0.02, -0.02]	[0.9093, -0.4161, 0.02, 0.9998]	[0.8493, -0.3761, 0.04, 0.9798]

If the same token appeared at positions 0 and 2, its two rows in $\mathbf{X}_0$ would now be **different** — that is exactly how the model can tell word order apart.

3. Batching Multiple Sequences (3D Tensors)

Transformers process multiple independent sentences in parallel to maximize hardware throughput (on a GPU in large frameworks; in this project NumPy runs the same batched matrix math on the CPU).

If we have two independent sentences:

- "The cat ate fish" → Shape: (4×512)
- "The fish ate cat" → Shape: (4×512)

Rather than concatenating them into a single 8×512 sequence, we stack them along a new axis called the **Batch Dimension**. This creates a 3D tensor of shape:

$$(Batch, Sequence, d_{\text{model}}) = (2, 4, 512)$$

- 2: Batch Size B (Number of independent sentences)
- 4: Sequence Length T (Tokens per sentence)
- 512: Vector dimension (d_{model})

The same $PE[:T]$ matrix of shape (T, d_{model}) is added to **every** sentence in the batch (NumPy broadcasting). Inside the attention layer, sentences never mix: each sentence only attends to its own tokens.

4. Padding Variable-Length Sequences

Matrix operations require uniform shapes and cannot process "ragged" batches where sentences have varying lengths. To enforce uniformity, the pipeline uses **Padding** on the token IDs:

1. **Truncate:** Any sentence longer than T_{max} (512 in this project) is cut to 512 tokens.
2. **Determine Max Length:** Identify the longest sequence in the current batch (e.g., 6 tokens). That becomes T for this batch.

3. **Append Filler Tokens:** Append the special `<pad>` token to shorter sentences until all sequences match the max length.

- **Sentence 1:** ["The", "cat", "ate", "fish", "[PAD]", "[PAD]"]
- **Sentence 2:** ["The", "cat", "ate", "[PAD]", "[PAD]", "[PAD]"]
- **Sentence 3:** ["The", "cat", "ate", "fish", "quickly", "today"]

This guarantees a perfectly uniform ID tensor of shape (3, 6), which becomes (3, 6, 512) after embedding.

Which ID is [PAD]? It depends on the tokenizer. Many tutorials use ID 0; **our BPE tokenizer uses `<pad>` = ID 1** (`<s>` = 0, `<pad>` = 1, `</s>` = 2, `<unk>` = 3, `<mask>` = 4). The code always reads it from `vocab.json` instead of hard-coding it.

5. Applying the Attention Mask

While padding satisfies shape requirements, [PAD] tokens are meaningless filler. The attention mechanism must not assign weights to them or allow them to influence real words.

Step A — binary mask (created with the padding). A mask of shape (B, T) marks real tokens with 1 (True) and padding with 0 (False):

$$\text{Attention Mask} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Step B — additive mask (used inside attention). The binary mask is **not** added directly. It is first converted: $1 \rightarrow 0$ and $0 \rightarrow -\infty$. This converted version is the **M** that is added to the attention scores right before softmax:

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 0 & 0 & -\infty & -\infty \\ \dots & & & & & \end{bmatrix} \quad \text{Softmax}(\text{Scores} + \mathbf{M})$$

Because $e^{-\infty} = 0$, the softmax output is exactly 0.0 for all padded positions.

Worked Example

Scores of one query against 4 keys, where the 4th key is [PAD]:

	key 1	key 2	key 3	key 4 ([PAD])
Scores	2.0	1.0	0.5	0.3
Softmax without mask	0.5638	0.2074	0.1258	0.1030 ✗
Scores + M	2.0	1.0	0.5	$-\infty$
Softmax with mask	0.6285	0.2312	0.1402	0.0000 ✓

Without the mask, 10% of the attention would be wasted on filler. With it, the weight is redistributed over the real tokens and still sums to 1.0.

In code, $-\infty$ is replaced by -10^9 : e^{-10^9} is also exactly 0.0 in floating point, but it avoids NaN in the corner case where every key in a row is masked.

Final Output: The resulting package — a uniform 3D tensor $\mathbf{X}_0$ of shape (B, T, d_{model}) carrying integrated positional signals, together with its (B, T) boolean attention mask — is delivered directly into the first Multi-Head Attention block.

Chapter 4

Multi-Head Self-Attention

In one sentence: every token asks a question (Query), every token offers a label (Key) and a payload (Value); each token's new vector is a weighted average of all Values, weighted by how well its Query matches each Key.

$X (B, T, d_{model}) \rightarrow Q, K, V \rightarrow \text{softmax}(QK^T/\sqrt{d_k} + M) \rightarrow \times V \rightarrow \text{concat heads} \rightarrow \times W_0 \rightarrow (B, T, d_{model})$

Code: `src/models/layers/multi_head_attention.py` $\rightarrow$ `MultiHeadAttention`

Phase 1: Data Ingestion (The Starting State)

The attention mechanism receives its input directly from the Positional Encoding pipeline.

At this exact moment, the input is a 3D tensor $\mathbf{X}$ with the shape (B, T, d_{model}) , where:

- **B (Batch Size):** The number of independent sentences being processed simultaneously.
- **T (Sequence Length):** The maximum number of tokens in the sequence (including padded tokens).
- **d_{model} (Embedding Dimension):** The vector size for each token (e.g., 512).

It also receives the (B, T) attention mask that marks which tokens are [PAD].

Every 512-dimensional vector inside this tensor contains the combined semantic meaning (from the embedding matrix) and physical location (from the sinusoidal positional encodings) of a single word. However, the words remain completely isolated; they have no context about the other words in their sequence.

(For blocks 2...N, $\mathbf{X}$ is simply the output of the previous encoder block — same shape.)

Phase 2: Single-Head Attention Pipeline

To build context, the Transformer uses the **Scaled Dot-Product Attention** mechanism. This allows every word to mathematically query the sequence, evaluate how much attention to pay to every other word, and update its own representation based on the results.

Step 1: Initialization of Weight Matrices

The mechanism relies on three learnable linear projection matrices. When the model is built, these are initialized with random values (we use Xavier/Glorot initialization: normal with $\sigma = \sqrt{2/(\text{fan}_{in} + \text{fan}_{out})}$):

- W_Q (**Query Weights**): Shape (d_{model}, d_k)
- W_K (**Key Weights**): Shape (d_{model}, d_k)
- W_V (**Value Weights**): Shape (d_{model}, d_v)

(Note: d_v is practically always equal to d_k . With a single head, d_k could be the full 512. With h heads, each head gets $d_k = d_{model}/h$ — that is where the value 64 comes from; see Phase 3.)

Step 2: Creating Q, K, and V

The input tensor $\mathbf{X}$ is multiplied by these three weight matrices to create three distinct contextual components for every token:

1. **Query ($\mathbf{Q} = \mathbf{X}W_Q$)**: What the token is looking for in the sequence.
2. **Key ($\mathbf{K} = \mathbf{X}W_K$)**: What the token advertises about its own meaning/grammar.
3. **Value ($\mathbf{V} = \mathbf{X}W_V$)**: The actual payload of semantic information the token will share if selected.

Each resulting tensor now has the shape (B, T, d_k) .

Step 3: The Scaled Dot-Product Math

The single-head attention formula is executed on these tensors:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

1. **Dot Product (Similarity Scores)**: The model calculates $\mathbf{Q}\mathbf{K}^T$. This produces a (B, T, T) grid of raw scores: entry $[t_1, t_2]$ says how strongly the Query of token t_1 matches the Key of token t_2 .
2. **Scaling**: The raw scores are divided by $\sqrt{d_k}$ (e.g., $\sqrt{64} = 8$). Dot products of long vectors produce large numbers that would push softmax into regions with near-zero gradients. Scaling keeps their variance around 1.
3. **Masking**: The **additive** mask $\mathbf{M}$ is added to the score grid: 0 for real tokens, $-\infty$ for padded keys (it is built from the binary 1/0 mask — see 2-positional_encoding.md, Section 5).
4. **Softmax**: The softmax function is applied across the last dimension (over keys) of the grid. Each row becomes weights that sum to 1.0. Because $e^{-\infty} = 0$, padded tokens get exactly 0.0 attention.
5. **Weighted Sum**: The (B, T, T) weight grid is multiplied by the Value tensor $\mathbf{V}$ of shape (B, T, d_k) .

Single-Head Output: A context-aware tensor of shape (B, T, d_k) . Token representations are no longer isolated; they are now weighted averages of the Values of all relevant words in the sequence.

Worked Example (one head, $T = 3$, $d_k = 2$, token 3 is [PAD])

Suppose the projections have already produced (one row per token):

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ 9 & 9 \end{bmatrix}$$

1. **Scores $\mathbf{QK}^T$** (row = query token, column = key token):

$$\mathbf{QK}^T = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix}$$

2. **Scale by $\sqrt{2} \approx 1.4142$:**

$$\frac{\mathbf{QK}^T}{\sqrt{2}} = \begin{bmatrix} 0.7071 & 0.7071 & 0 \\ 0 & 0.7071 & 1.4142 \\ 0.7071 & 1.4142 & 1.4142 \end{bmatrix}$$

3-4. **Mask column 3, then softmax each row:**

$$\text{weights} = \begin{bmatrix} 0.5000 & 0.5000 & 0 \\ 0.3302 & 0.6698 & 0 \\ 0.3302 & 0.6698 & 0 \end{bmatrix}$$

Row 2 would have preferred the [PAD] key (score 1.4142), but the mask forces that weight to 0.

5. **Weighted sum with V:**

$$\text{weights} \cdot \mathbf{V} = \begin{bmatrix} 0.5 \cdot [1, 2] + 0.5 \cdot [3, 0] \\ 0.3302 \cdot [1, 2] + 0.6698 \cdot [3, 0] \\ \dots \end{bmatrix} = \begin{bmatrix} 2.0000 & 1.0000 \\ 2.3395 & 0.6605 \\ 2.3395 & 0.6605 \end{bmatrix}$$

Notice the [PAD] Value [9, 9] never leaks into any output.

Phase 3: Multi-Head Attention (MHA) Pipeline

Language is complex. A word often needs to track multiple relationships simultaneously (e.g., subject-verb agreement, emotional tone, and direct objects). A single head forces the network to average all these distinct needs into one Query. Multi-Head Attention solves this by splitting the vector space so the model can track multiple independent relationships in parallel.

Step 1: The Dimension Split

Instead of using one massive attention operation, the Transformer configures h independent "heads" (e.g., $h = 8$). To keep the computational cost identical to single-head attention, the model divides the embedding dimension by the number of heads:

$$d_k = \frac{d_{model}}{h} = \frac{512}{8} = 64$$

Step 2: Unique Weight Initialization

The model initializes 8 entirely distinct sets of weight matrices, each of shape $(512, 64)$:

- $W_Q^1 \dots W_Q^8$
- $W_K^1 \dots W_K^8$
- $W_V^1 \dots W_V^8$

Because each of these 24 matrices is initialized with completely different random values, symmetry is broken. As the network trains, Head 1 might learn to track grammatical structure, while Head 2 explores rhyming or semantic groupings.

How the code stores them: the 8 matrices $W_Q^1 \dots W_Q^8$ (each 512×64) are placed side by side into one 512×512 matrix W_Q . Columns 0-63 belong to head 1, columns 64-127 to head 2, and so on. The math is identical, but one big matrix multiply replaces 24 small ones.

Step 3: Parallel Execution

The input tensor $\mathbf{X}$ ($B, T, 512$) is projected into 8 independent sets of $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i$ matrices. In code this is done by one projection followed by a reshape:

Step	Shape	Example ($B = 4, T = 512$)
$\mathbf{X}W_Q$	$(B, T, 512)$	$(4, 512, 512)$
split last axis into 8 heads of 64	$(B, T, 8, 64)$	$(4, 512, 8, 64)$
move the head axis forward $\rightarrow \mathbf{Q}$	(B, h, T, d_k)	$(4, 8, 512, 64)$
scores $\mathbf{QK}^T$	(B, h, T, T)	$(4, 8, 512, 512)$
weights $\times \mathbf{V}$	(B, h, T, d_k)	$(4, 8, 512, 64)$

The Scaled Dot-Product math detailed in Phase 2 runs entirely independently for each of the 8 heads at the same time (the head axis behaves just like an extra batch axis).

Output of the Parallel Execution: 8 distinct tensors, each with the shape $(B, T, 64)$ — stored together as one $(B, 8, T, 64)$ tensor.

Step 4: Concatenation

To synthesize the parallel insights back into a single vector per token, the model concatenates the 8 output tensors side-by-side along the vector feature dimension:

$$\text{Concat}(\text{head}_1, \dots, \text{head}_8)$$

Gluing eight 64-dimensional vectors together restores the sequence to its original 512-dimensional width. The resulting tensor shape is $(B, T, 512)$. (In code: the reverse of the reshape above, $(B, 8, T, 64) \rightarrow (B, T, 8, 64) \rightarrow (B, T, 512)$.)

Step 5: The Final Linear Projection

Simply concatenating the vectors leaves the information segregated (e.g., dimensions 0-63 only contain Head 1's data). To fuse these separated subspaces into a cohesive representation, the concatenated tensor is multiplied by a final, learnable output weight matrix, W_O .

- W_O **Shape:** $(h \cdot d_k, d_{model}) = (512, 512)$

$$\mathbf{Output} = \text{Concat}(\text{head}_1, \dots, \text{head}_8) \cdot W_O$$

Final Block Output: The Multi-Head Attention mechanism outputs a single tensor of shape (B, T, d_{model}) . This shape matches the tensor that originally entered the block, but its contents are transformed. It is now ready to be passed directly into the Add & Norm residual connection.

Parameter Count (per encoder block)

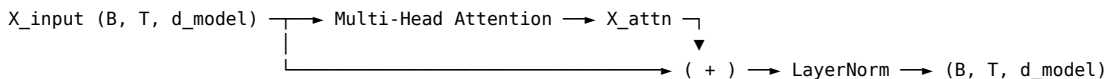
Weight	Shape	Numbers
W_Q (8 heads combined)	512×512	262,144
W_K	512×512	262,144
W_V	512×512	262,144
W_O	512×512	262,144
Total		1,048,576

There are no bias vectors in our attention layer.

Chapter 5

Add & Norm (after Multi-Head Attention)

In one sentence: add the attention block's input back to its output ("Add"), then rescale every token vector to mean 0 and variance 1 ("Norm").



This is the **Post-LN** arrangement from the original Transformer paper (normalize *after* adding), which is what this project uses.

Code: `src/models/layers/add_and_norm.py` → `ResidualConnection` (Add), `LayerNormalization` (Norm), `AddNormBlock` (both together)

Phase 1: The "Add" Component (Residual Connections)

The "Add" step is a mathematical shortcut known as a residual connection (or skip connection). It takes the original tensor that entered the Multi-Head Attention block and adds it directly to the tensor that just came out of it.

If we denote the original input tensor as $\mathbf{X}_{input}$ and the output of the attention mechanism as $\text{MHA}(\mathbf{X}_{input})$, the operation is strict element-wise addition:

$$\mathbf{X}_{res} = \mathbf{X}_{input} + \text{MHA}(\mathbf{X}_{input})$$

Both tensors have shape (B, T, d_{model}) , so the addition is position-by-position, number-by-number. No weights are involved.

Why Residual Connections are Critical

1. **Solving the Vanishing Gradient Problem:** In deep neural networks, multiplying gradients across dozens of layers during backpropagation causes the numbers to shrink towards zero, freezing the early layers (like the embedding layer) from learning. Residual connections act as an unimpeded highway: the derivative of $a + b$ with respect to a is exactly 1, so gradients flow backwards through the addition without shrinking (see 9-backpropagation.md).

2. **Preserving Original Identity:** The Multi-Head Attention block aggressively remixes vectors to inject context. While context is crucial, a word must not completely “forget” what it actually is. Adding the input vector back into the contextualized vector guarantees that the token's core semantic identity and positional location are preserved.
-

Phase 2: The “Norm” Component (Layer Normalization)

Once the tensors are added together, the resulting values can become large or unstable. To standardize these values, the Transformer applies Layer Normalization.

Why Layer Normalization Over Batch Normalization?

In computer vision, networks use Batch Normalization, which calculates the mean and variance across an entire batch of images. This fails in NLP because sentences have highly variable sequence lengths and rely heavily on [PAD] tokens.

Layer Normalization ignores the batch and sequence length entirely. Instead, it calculates the mean and variance independently for **every single token** across its own vector dimensions (d_{model}).

Batch Norm: statistics taken DOWN the batch (same feature, many examples)

Layer Norm: statistics taken ACROSS one token's own 512 features ← used here

The Mathematical Formulation

For every single token vector $\mathbf{x} \in \mathbb{R}^{d_{model}}$ inside the sequence, the layer computes:

1. **Mean (μ):** The average value of the 512 numbers inside that specific token's vector.

$$\mu = \frac{1}{d_{model}} \sum_{j=1}^{d_{model}} x_j$$

2. **Variance (σ^2):** How spread out those 512 numbers are from the mean.

$$\sigma^2 = \frac{1}{d_{model}} \sum_{j=1}^{d_{model}} (x_j - \mu)^2$$

3. **Normalization & Scaling:** The vector is normalized and then scaled/shifted using two learnable parameter vectors, γ (scale) and β (shift), each of shape (d_{model}) and optimized during training.

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

(Notes: ϵ is a tiny number like 10^{-5} that prevents division by zero. $\odot$ means element-wise multiplication. At initialization $\gamma = [1, 1, \dots, 1]$ and $\beta = [0, 0, \dots, 0]$, so the layer starts as a pure normalization; training can later learn to rescale or shift individual features.)

Worked Example ($d_{model} = 4$, one token)

	dim 1	dim 2	dim 3	dim 4
$\mathbf{X}_{input}$	1.0	2.0	3.0	4.0
$\mathbf{X}_{attn}$ (MHA output)	0.5	-1.0	1.0	1.5
Add: $\mathbf{X}_{res}$	1.5	1.0	4.0	5.5

Norm:

1. Mean: $\mu = (1.5 + 1.0 + 4.0 + 5.5)/4 = 12/4 = 3.0$
2. Variance: $\sigma^2 = ((-1.5)^2 + (-2.0)^2 + 1.0^2 + 2.5^2)/4 = (2.25 + 4 + 1 + 6.25)/4 = 3.375$
3. Standard deviation: $\sqrt{3.375 + 0.00001} \approx 1.8371$
4. Normalize: $(\mathbf{x} - 3.0)/1.8371$

	dim 1	dim 2	dim 3	dim 4
$\mathbf{x} - \mu$	-1.5	-2.0	1.0	2.5
$\hat{\mathbf{x}} = (\mathbf{x} - \mu)/1.8371$	-0.8165	-1.0887	0.5443	1.3608
$\gamma \odot \hat{\mathbf{x}} + \beta$ (with $\gamma = 1, \beta = 0$)	-0.8165	-1.0887	0.5443	1.3608

Check: the output has mean 0 and variance 1. The *pattern* (dim 4 largest, dim 2 smallest) is kept; only the scale is standardized.

Phase 3: The Complete Add & Norm Pipeline

When the (B, T, d_{model}) tensor exits the Multi-Head Attention block, the Add & Norm sequence executes:

1. **The Inputs:** Two tensors are held in memory:
 - $\mathbf{X}_{input}$: The $(B, T, 512)$ tensor that entered this encoder block. For **block 1** this is Word Embeddings + Positional Encodings ($\mathbf{X}_0$); for **later blocks** it is the output of the previous block.
 - $\mathbf{X}_{attn}$: The new $(B, T, 512)$ tensor produced by the Multi-Head Attention block.
2. **Element-Wise Addition:** They are added index by index. If the 3rd word in the sentence has a value of 0.44 at dimension 12 in $\mathbf{X}_{input}$, and a value of 1.12 in $\mathbf{X}_{attn}$, the new value is 1.56. The resulting tensor $\mathbf{X}_{res}$ keeps the exact shape $(B, T, 512)$.
3. **Token-by-Token Normalization:** For every token in the batch ($B \times T$ tokens), the algorithm scans its 512 features, calculates the mean and variance, centers the values around 0, and applies the learnable γ and β weights.
4. **Final Output:** The block outputs a stable, contextualized, and normalized 3D tensor of shape (B, T, d_{model}) , called $\mathbf{X}_{norm1}$. It becomes the input of the Feed-Forward Network.

This entire Add & Norm sequence acts as a stabilizing wrapper. In the standard Transformer architecture, this wrapper is used twice per encoder block: once immediately after

the Multi-Head Attention (this document), and a second time immediately after the Feed-Forward Network (6-second-add-and-norm.md). The two uses have **separate** γ, β parameters.

Learnable parameters here: $\gamma_1 (512) + \beta_1 (512) = 1,024$ numbers per block.

Chapter 6

Position-wise Feed-Forward Network (FFN)

The Position-wise Feed-Forward Network (FFN) sits immediately after the first Add & Norm layer in the Transformer block. While the Multi-Head Attention layer is responsible for routing information *between* different words in a sentence, the FFN is responsible for processing each word's new contextualized vector *individually* to extract deeper semantic features.

In the attention layer, tokens "talk" to each other. In the feed-forward layer, tokens process what they just learned in complete isolation.

$X_{\text{norm1}} (B, T, 512) \xrightarrow[\text{expand}]{\times W_1 + b_1} (B, T, 2048) \xrightarrow[\text{gate}]{\text{ReLU}} (B, T, 2048) \xrightarrow[\text{compress}]{\times W_2 + b_2} (B, T, 512)$

Code: `src/models/layers/feed_forward.py` → `PositionwiseFeedForward`

1. The Core Architecture

The FFN consists of two standard linear transformations (dense layers) with a non-linear activation function placed between them. It is applied to each position (each token) separately and identically — the **same** W_1, b_1, W_2, b_2 are used for every token in every sentence. That is what "position-wise" means.

The mathematical formulation for a single token vector $\mathbf{x}$ (a row of length d_{model}) is:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$$

- W_1 **and** b_1 : The weights and biases for the first linear layer.
- $\max(0, z)$: The ReLU (Rectified Linear Unit) activation function, applied to each number separately. Modern models often swap this for GELU, but the architectural principle remains identical.
- W_2 **and** b_2 : The weights and biases for the second linear layer.

Initialization in this project: $W_1, W_2 \sim \mathcal{N}(0, \sigma = 0.02)$; b_1, b_2 start at zero.

2. The Expansion and Compression Strategy

The Transformer does not keep the vector dimension static through this network. It uses an expansion-then-compression strategy to give the network mathematical space to learn

complex feature representations.

If the input vector dimension is d_{model} (e.g., 512), the inner layer dimension, denoted as d_{ff} , is typically scaled up by a factor of 4 (e.g., $d_{ff} = 2048$).

Step 1: The Up-Projection (W_1)

The contextualized (B, T, d_{model}) tensor enters the FFN. Every single word vector is multiplied by W_1 and b_1 is added.

- **W_1 Shape:** $(d_{model}, d_{ff}) \rightarrow (512, 2048)$; **b_1 Shape:** (2048)
- **Tensor Shape Output:** $(B, T, 2048)$
- **Purpose:** The network projects the 512-dimensional vector into a much larger 2048-dimensional space. This allows the model to disentangle complex combinations of features that were mixed together during the attention phase.

Step 2: The Non-Linearity (ReLU)

The network applies the ReLU function to the $(B, T, 2048)$ tensor, converting any negative number to 0 and leaving positive numbers untouched.

- **Purpose:** Without this non-linear function, the two linear layers (W_1 and W_2) would mathematically collapse into a single linear layer ($\mathbf{x}W_1W_2 = \mathbf{x}W'$), stripping the network of its deep-learning capabilities. ReLU acts as a gate, discarding irrelevant features (turning them to zero) and allowing strong signals to pass through.

Step 3: The Down-Projection (W_2)

The activated $(B, T, 2048)$ tensor is multiplied by W_2 and b_2 is added.

- **W_2 Shape:** $(d_{ff}, d_{model}) \rightarrow (2048, 512)$; **b_2 Shape:** (512)
- **Tensor Shape Output:** $(B, T, 512)$
- **Purpose:** The network takes the filtered information from the 2048-dimensional space and compresses it back down into the standard 512-dimensional space required by the rest of the Transformer.

3. Worked Example (one token, $d_{model} = 2$, $d_{ff} = 4$)

(Here the expansion is $\times 2$ instead of $\times 4$ just to keep the numbers small.)

$$\mathbf{x} = [1, -1], \quad W_1 = \begin{bmatrix} 1 & -1 & 0.5 & 2 \\ 0 & 1 & -0.5 & 1 \end{bmatrix}, \quad b_1 = [0, 0, 0, -0.5]$$

$$W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 2 \end{bmatrix}, \quad b_2 = [0.1, 0.1]$$

Step 1 — expand: each hidden unit k is $x_1 W_1[1, k] + x_2 W_1[2, k] + b_1[k]$:

unit	calculation	hidden
1	$1 \cdot 1 + (-1) \cdot 0 + 0$	1.0
2	$1 \cdot (-1) + (-1) \cdot 1 + 0$	-2.0
3	$1 \cdot 0.5 + (-1) \cdot (-0.5) + 0$	1.0
4	$1 \cdot 2 + (-1) \cdot 1 - 0.5$	0.5

Step 2 — ReLU: $[1.0, -2.0, 1.0, 0.5] \rightarrow [1.0, \mathbf{0.0}, 1.0, 0.5]$ (unit 2 is switched off)

Step 3 — compress: $[1, 0, 1, 0.5] \cdot W_2 + b_2$

- output 1: $1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0.5 \cdot (-1) + 0.1 = 1.6$
- output 2: $1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 + 0.5 \cdot 2 + 0.1 = 2.1$

$$\text{FFN}([1, -1]) = [1.6, 2.1]$$

Same width in and out ($2 \rightarrow 4 \rightarrow 2$), just like $512 \rightarrow 2048 \rightarrow 512$ in the real model.

4. The Final Tensor Flow

Tracking a batched tensor through this specific block:

1. **Input:** A stable, contextualized tensor arrives from the first Add & Norm layer: $(B, T, 512)$.
2. **Linear 1** (W_1, b_1): Expands the tensor to $(B, T, 2048)$.
3. **Activation:** Applies ReLU independently to every number in the tensor. The shape remains $(B, T, 2048)$.
4. **Linear 2** (W_2, b_2): Compresses the tensor back to $(B, T, 512)$.
5. **Output:** The FFN outputs a transformed tensor of shape $(B, T, 512)$.

Once the tensor exits the Feed-Forward Network, it is immediately routed into a **second Add & Norm layer**. The original $(B, T, 512)$ input to the FFN is added to the FFN's output (Residual Connection), and the result is normalized. This completes one entire Encoder block. This block is then repeated sequentially to build deeper representations — 12 times in BERT-base, **6 times in this project**.

Parameter Count (per encoder block)

Parameter	Shape	Numbers
W_1	512×2048	1,048,576
b_1	2048	2,048
W_2	2048×512	1,048,576
b_2	512	512
Total		2,099,712

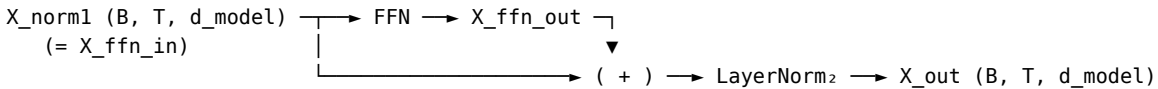
The FFN holds about two-thirds of each block's parameters.

Chapter 7

Second Add & Norm (after the Feed-Forward Network)

The second **Add & Norm** layer sits directly after the Position-wise Feed-Forward Network (FFN), forming the final component of a complete Transformer Encoder block.

It uses the exact same mathematical formulation and principles as the first Add & Norm layer (4-add_and_normalization.md), acting as a stabilizing wrapper for the transformations executed by the FFN. The only differences are **what** gets added and that it has its **own** γ_2, β_2 .



Code: the second AddNormBlock inside EncoderBlock (src/models/encoder.py, attribute add_norm_2)

1. The "Add" Component (Second Residual Connection)

The input tensor that was fed into the FFN — which we call $\mathbf{X}_{ffn_in}$, shape (B, T, d_{model}) — is the output of the **first** Add & Norm ($\mathbf{X}_{norm1}$). It is kept in memory. After the FFN processes it through its $d_{model} \rightarrow d_{ff} \rightarrow d_{model}$ expansion and compression, the output $\text{FFN}(\mathbf{X}_{ffn_in})$ is added directly back to it:

$$\mathbf{X}_{res2} = \mathbf{X}_{ffn_in} + \text{FFN}(\mathbf{X}_{ffn_in})$$

Why it is applied here

- **Information Highway:** The FFN applies heavy non-linear filtering (ReLU) which discards information in the 2048-dimensional space. The skip connection ensures that the contextualized attention information from the earlier stage is not accidentally destroyed.
- **Gradient Preservation:** During backpropagation, the loss gradient can flow directly through this addition back into the attention layers and embedding matrix without being diminished by the FFN's weights (W_1, W_2).

2. The “Norm” Component (Second Layer Normalization)

The tensor $\mathbf{X}_{res2}$ is immediately passed into a second, independent Layer Normalization.

This block maintains its own distinct set of learnable parameters: γ_2 (scale) and β_2 (shift), both of shape (d_{model}) . They start as $\gamma_2 = 1$, $\beta_2 = 0$ and are trained separately from γ_1, β_1 .

For every single token vector $\mathbf{x} \in \mathbb{R}^{d_{model}}$ across the batch and sequence:

1. Calculate Token Mean (μ):

$$\mu = \frac{1}{d_{model}} \sum_{j=1}^{d_{model}} x_j$$

2. Calculate Token Variance (σ^2):

$$\sigma^2 = \frac{1}{d_{model}} \sum_{j=1}^{d_{model}} (x_j - \mu)^2$$

3. Normalize, Scale, and Shift:

$$\text{LayerNorm}_2(\mathbf{x}) = \gamma_2 \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_2$$

3. Worked Example (continuing the FFN example, $d_{model} = 2$)

From 5-feed_forward.md: $\mathbf{X}_{ffn_in} = [1, -1]$ and $\text{FFN}(\mathbf{X}_{ffn_in}) = [1.6, 2.1]$.

1. **Add:** $\mathbf{X}_{res2} = [1 + 1.6, -1 + 2.1] = [2.6, 1.1]$
2. **Mean:** $\mu = (2.6 + 1.1)/2 = 1.85$
3. **Variance:** $\sigma^2 = (0.75^2 + (-0.75)^2)/2 = 0.5625$, so $\sqrt{\sigma^2 + \epsilon} \approx 0.75$
4. **Normalize:** $[(2.6 - 1.85)/0.75, (1.1 - 1.85)/0.75] = [1.0, -1.0]$
5. **Scale & shift** ($\gamma_2 = 1$, $\beta_2 = 0$): $\mathbf{X}_{out} = [1.0, -1.0]$

(With only 2 features the normalized values are always ± 1 ; with 512 features you get a full spread of values, as in the example in 4-add_and_normalization.md.)

4. Top-to-Bottom Flow of the Second Add & Norm Layer

1. Input Tensors in Memory:

- $\mathbf{X}_{ffn_in}$: The normalized tensor coming out of the *first* Add & Norm block — shape $(B, T, 512)$.
- $\mathbf{X}_{ffn_out}$: The tensor produced by the Feed-Forward Network ($W_1 \rightarrow \text{ReLU} \rightarrow W_2$) — shape $(B, T, 512)$.

2. Element-Wise Addition:

- Compute $\mathbf{X}_{res2} = \mathbf{X}_{ffn_in} + \mathbf{X}_{ffn_out}$.

- Output shape remains $(B, T, 512)$.

3. Layer Normalization:

- For every token $t \in [1, T]$ in every sentence $b \in [1, B]$, the 512 feature values are normalized to have zero mean and unit variance, then rescaled via γ_2 and shifted via β_2 .

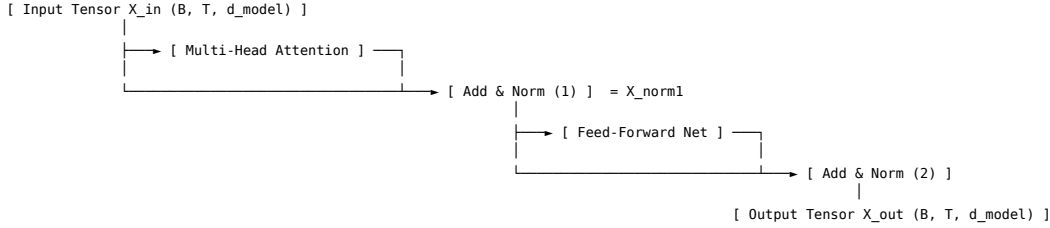
4. Final Block Output:

- The layer outputs a clean, stable 3D tensor of shape (B, T, d_{model}) .

Learnable parameters here: $\gamma_2 (512) + \beta_2 (512) = 1,024$ numbers per block.

Summary of the Entire Encoder Block Cycle

With the completion of the second Add & Norm layer, the data tensor has traveled through one complete Transformer Encoder block:



Sub-layer	Learnable parameters	Count
Multi-Head Attention	W_Q, W_K, W_V, W_O	1,048,576
Add & Norm 1	γ_1, β_1	1,024
Feed-Forward	W_1, b_1, W_2, b_2	2,099,712
Add & Norm 2	γ_2, β_2	1,024
One encoder block		3,150,336

The output tensor has the exact same dimensions (B, T, d_{model}) as the tensor that entered the block. Because the shape is preserved, this output can be fed directly into the **next** Encoder block in the stack (Block 2 of 6 in this project) to build deeper abstract representations. Each block has its own, separately learned weights.

Chapter 8

Linearization (LM Head) and Softmax

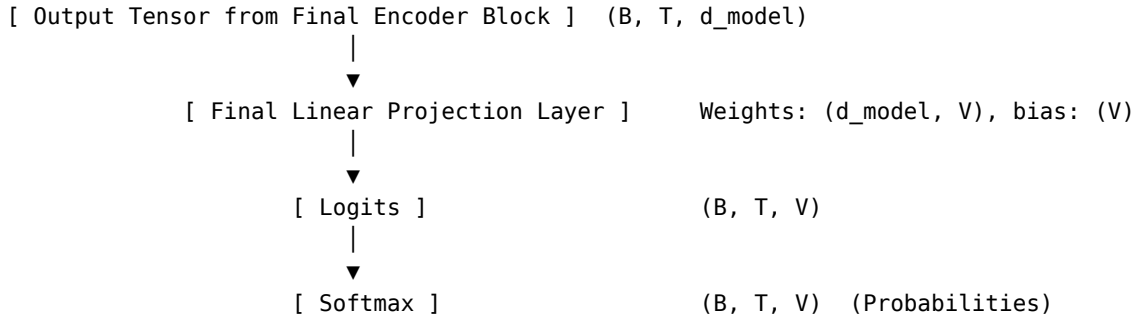
In one sentence: a final linear layer turns each 512-number token vector into one score per vocabulary word, and softmax turns those scores into probabilities.

Code: `src/models/layers/lm_head.py` → `LanguageModelingHead`; loss in `src/training/loss.py`

1. The Linearization and Softmax Head

After passing through N repeated Encoder blocks (6 in this project; 12 or 24 in larger models), the tensor exiting the final block has the shape (B, T, d_{model}) .

To convert these hidden representations back into predictions over real words (vocabulary tokens), the model routes the tensor through a **Language Modeling Head** (or Classification Head).



Step 1: The Linear Projection (Un-embedding)

The model passes the (B, T, d_{model}) tensor through a standard Dense (Linear) layer that projects the vector size from d_{model} (512) directly to the size of the entire vocabulary V .

$$\mathbf{Z} = \mathbf{X}_{final} \cdot \mathbf{W}_{LM} + b_{LM}$$

	General example	This project
Input $\mathbf{X}_{final}$	$(B, T, 512)$	$(4, 512, 512)$
$\mathbf{W}_{LM}$	$(512, 50000)$	$(512, 30000)$

	General example	This project
b_{LM}	(50000)	(30000)
Output $\mathbf{Z}$ (logits)	$(B, T, 50000)$	$(4, 512, 30000)$

The output values inside $\mathbf{Z}$ are called **Logits**. Every token position t in every sentence now has an unnormalized score for every possible token in the vocabulary. Logits can be any real number (negative, zero, positive).

(Note on Weight Tying: In many architectures, W_{LM} is the **transposed Input Embedding matrix** E^T — shape $(512, V)$ — instead of a separate matrix. Reusing the input embedding weights cuts the parameter count and can improve stability. The code supports it via `tie_weights`; this project's config has it switched **off**.)

Step 2: The Softmax Layer

To convert these raw logit scores into probabilities, the model applies the Softmax function along the vocabulary dimension (V), separately for every position:

$$P(w_i) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

- **Input Shape:** (B, T, V)
- **Output Shape:** (B, T, V)

For every position in the sequence, the V values now sum to 1.0 (100%). The token with the highest probability is the model's prediction.

Numerical safety: e^z overflows for large z . The code subtracts the row maximum first, $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - \max \mathbf{z})$, which gives the identical result without overflow.

Worked Example (one position, toy vocabulary of $V = 4$)

token	logit z_i	e^{z_i}	probability $P(w_i)$
"नेपाल"	2.0	7.389	0.6381
"सुन्दर"	1.0	2.718	0.2347
"देश"	0.1	1.105	0.0954
"हो"	-1.0	0.368	0.0318
sum		11.580	1.0000

Each probability is $e^{z_i}/11.580$. The model predicts "नेपाल".

2. What Is the Target? (Masked Language Modelling)

This project's summarizer does not use MLM. It trains the encoder as a prefix-LM: the input is `<s> article </s> summary </s>`, and the target at each

summary position is the next summary token. See **“Project-Specific Training and Target Data Preparation Strategy”** in 8-end-to-end.md. MLM is explained below because it is the classic encoder objective and was this project's earlier pre-training setup. The “score only N positions” idea carries over unchanged.

The loss compares the predicted probabilities with a **ground-truth token**. But an encoder sees the whole sentence at once — if we asked it to predict the token at position t while that token is visible, it could simply copy its input and learn nothing.

The standard solution for encoders (used by BERT) is **Masked Language Modelling (MLM)**:

1. Pick a random $\sim 15\%$ of the real (non-[PAD]) tokens in the batch.
2. Replace them in the input with the special <mask> token (ID 4 in our vocabulary).
3. Run the encoder. Because of self-attention, the vector at a masked position can gather clues from the surrounding words.
4. Compute the loss **only at the masked positions**, with the **original** token as the target.

Original: नेपाल सुन्दर देश हो
 Input: नेपाल <mask> देश हो
 Target: - सुन्दर - - ← loss only here

Because only masked positions matter, the code runs the LM head on just those N vectors: $(N, 512) \rightarrow (N, V)$ instead of $(B, T, 512) \rightarrow (B, T, V)$. With $B = 4$, $T = 512$ and 15% masking, $N \approx 307$ instead of 2048 positions — about 85% less work, same learning signal. In the summarizer, the N positions are the summary positions instead, at most $B \times 256 = 1024$ per batch.

Cross-Entropy Loss

For each scored position (masked token in MLM, summary token in the summarizer), the loss is the negative log of the probability the model gave to the correct token; the batch loss is the average over the N scored positions:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \log P(\text{target}_n)$$

With the toy example above:

- If the true token is “नेपाल” ($P = 0.6381$): $\mathcal{L} = -\ln 0.6381 = 0.449$ (good prediction, small loss)
- If the true token is “देश” ($P = 0.0954$): $\mathcal{L} = -\ln 0.0954 = 2.349$ (bad prediction, large loss)

Sanity check for this project: an untrained model spreads probability evenly, $P \approx 1/30000$, so the first loss should be close to $\ln 30000 \approx 10.31$.

3. Initial Guesses vs. Actual Training (Updating Embeddings)

Where Does Actual Training Happen?

Training happens through **End-to-End Backpropagation**:

1. **Forward Pass:** The input token IDs pass through the initial Embeddings → Positional Encodings → Attention Blocks → FFNs → Linear Projection → Softmax.
2. **Loss Calculation:** The model compares its predicted probability distribution against the true (masked) tokens using **Cross-Entropy Loss**.
3. **Backward Pass (Backpropagation):** The loss produces an error gradient ($\nabla \mathcal{L}$). This gradient travels **all the way back** through every single layer from the top down:

Loss → Linear Layer → Block N → ... → Block 1 → Positional Addition → Input Embeddings

4. **Optimizer Step:** Every weight matrix in the entire pipeline is updated using its gradient. The simplest rule is plain gradient descent:

$$W_{new} = W_{old} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

This project uses **Adam**, which adapts the step size per weight (AdamW is a variant with decoupled weight decay). Every formula — for every weight and bias — is worked out in `9-backpropagation.md`.

Do Input Embeddings Update After the First Round?

Yes! Input embeddings are continuously updated throughout the entire training process.

1. Initial State (Step 0 of Training):

- The Input Embedding matrix E of shape (V, d_{model}) is initialized with random numbers (from $\mathcal{N}(0, \sigma = 0.02)$).
- At step 0, the word "cat" and the word "dog" have completely random, meaningless vectors.

2. During Training (Many Iterations):

- In every training step, gradients flow directly into the rows of the embedding matrix E corresponding to the tokens in that batch.
- If "cat" and "dog" repeatedly appear in similar contextual positions across many sentences, backpropagation pushes their 512-dimensional vectors closer together in coordinate space.

3. Inference State (Deployment / Post-Training):

- Once training is complete, all weights—including the Input Embedding matrix, $W_Q, W_K, W_V, W_O, W_1, W_2$, and the Linear Head—are **frozen**.
- When you run a trained model, the embeddings are static, fully optimized lookup vectors that no longer change.

Parameter Count

Parameter	Shape	Numbers
W_{LM}	512×30000	15,360,000
b_{LM}	30000	30,000
Total		15,390,000 (only b_{LM} if weights are tied)

Chapter 9

End-to-End Encoder Pipeline

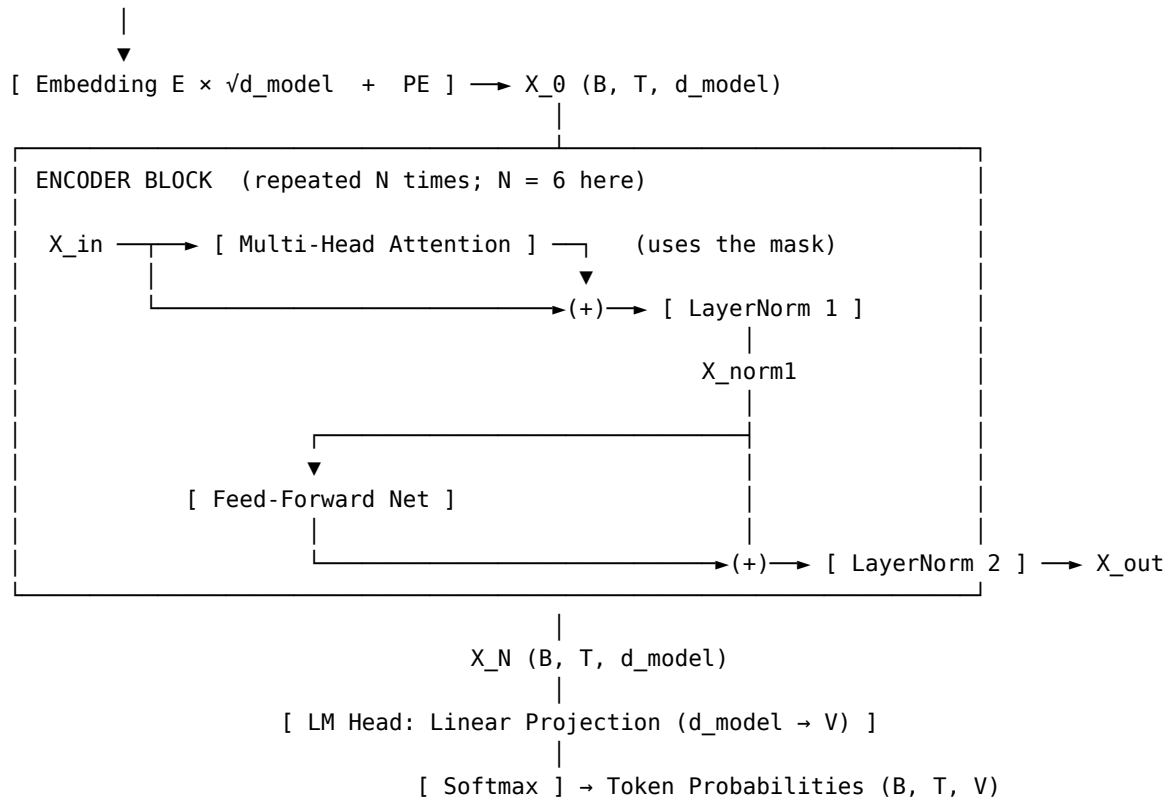
This document puts documents 1-7 together into one picture, with the exact shapes and numbers used in this project.

One distinction first: **within a single forward pass**, you do not re-fetch or re-add the raw input embeddings at later blocks; **across training steps (rounds)**, the weight values inside the Input Embedding matrix E are continuously updated via backpropagation.

Code: `src/models/encoder.py` → `EncoderBlock` (one block) and `TransformerEncoder` (the whole model)

Complete End-to-End Transformer Encoder Pipeline

Token IDs (B, T) + attention mask (B, T)



Shapes at Every Stage (this project: $B = 4, T = 512, d_{model} = 512, h = 8, d_k = 64, d_{ff} = 2048, V = 30000$)

Stage	Tensor	Shape	Example
Input	token IDs, mask	(B, T)	$(4, 512)$
Embedding	$E[\text{ids}] \cdot \sqrt{512}$	(B, T, d_{model})	$(4, 512, 512)$
Positional encoding	$PE[: T]$	(T, d_{model})	$(512, 512)$
Block input	$\mathbf{X}_0$	(B, T, d_{model})	$(4, 512, 512)$
Attention	$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	(B, h, T, d_k)	$(4, 8, 512, 64)$
Attention	scores / weights	(B, h, T, T)	$(4, 8, 512, 512)$
Attention	concatenated heads	(B, T, d_{model})	$(4, 512, 512)$
FFN	hidden	(B, T, d_{ff})	$(4, 512, 2048)$
Block output	$\mathbf{X}_{out}$	(B, T, d_{model})	$(4, 512, 512)$
Encoder output	$\mathbf{X}_N$	(B, T, d_{model})	$(4, 512, 512)$
LM head	logits / probabilities	(B, T, V)	$(4, 512, 30000)$

Stage 1: Input Embedding & Positional Encoding

1. Vocabulary Lookup

Given a batch of tokenized, padded sentences of shape (B, T) , each token ID is looked up in the **Input Embedding Matrix** $E \in \mathbb{R}^{V \times d_{model}}$ (here $V = 30000, d_{model} = 512$).

- **Scaling:** The retrieved vectors are multiplied by $\sqrt{d_{model}} \approx 22.63$ so that the embeddings are not dominated by the positional encodings.
- **Output Shape:** (B, T, d_{model})

2. Positional Encoding Addition

Because attention operates on all tokens simultaneously without inherent sequence order, fixed sinusoidal positional encodings (PE) are added directly to the embeddings:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$\mathbf{X}_0 = (E[\text{tokens}] \cdot \sqrt{d_{model}}) + PE[: T]$$

- **Operation:** Direct element-wise addition (the same PE rows are added to every sentence in the batch).
- **Output Shape:** $\mathbf{X}_0 \in \mathbb{R}^{B \times T \times d_{model}}$

Stage 2: The Stacked Encoder Blocks (1 to N)

The tensor $\mathbf{X}_0$ enters the first of N identical (in structure, not in weights) Encoder blocks. Each block consists of two main sub-layers: **Multi-Head Attention** and a **Feed-Forward Network**, both wrapped in **Add & Norm** residual connections.

1. Multi-Head Attention (MHA)

- **Projection:** $\mathbf{X}_{in}$ is multiplied by h independent sets of learnable weights ($W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{model} \times d_k}$, where $d_k = d_{model}/h = 64$) to generate Queries ($\mathbf{Q}_i$), Keys ($\mathbf{K}_i$), and Values ($\mathbf{V}_i$).

- **Scaled Dot-Product Attention:**

$$\text{head}_i = \text{softmax} \left(\frac{\mathbf{Q}_i \mathbf{K}_i^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}_i$$

(where $\mathbf{M}$ is the additive mask: 0 for real tokens, $-\infty$ for [PAD] keys)

- **Concatenation & Output Projection:**

$$\text{MHA}(\mathbf{X}_{in}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$$

(where $W_O \in \mathbb{R}^{d_{model} \times d_{model}}$)

- **Shape:** Transforms $(B, T, d_{model}) \rightarrow (B, T, d_{model})$.

2. First Add & Norm

- **Residual Connection:** Adds the block input $\mathbf{X}_{in}$ directly to the attention output to prevent vanishing gradients and preserve token identity:

$$\mathbf{X}_{res1} = \mathbf{X}_{in} + \text{MHA}(\mathbf{X}_{in})$$

- **Layer Normalization:** Calculates mean μ and variance σ^2 across the d_{model} features for each token independently:

$$\mathbf{X}_{norm1} = \gamma_1 \odot \frac{\mathbf{X}_{res1} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_1$$

- **Shape:** (B, T, d_{model})

3. Position-Wise Feed-Forward Network (FFN)

- **Up-Projection:** Expands each token vector individually from d_{model} to d_{ff} ($4 \times d_{model} = 2048$):

$$\mathbf{H} = \text{ReLU}(\mathbf{X}_{norm1} W_1 + b_1) \quad \text{where } W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$$

- **Down-Projection:** Compresses back to model dimension:

$$\text{FFN}(\mathbf{X}_{norm1}) = \mathbf{H} W_2 + b_2 \quad \text{where } W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$$

- **Shape:** $(B, T, d_{model}) \rightarrow (B, T, d_{ff}) \rightarrow (B, T, d_{model})$

4. Second Add & Norm

- **Residual Connection:** Adds the input of the FFN ($\mathbf{X}_{norm1}$) to its output:

$$\mathbf{X}_{res2} = \mathbf{X}_{norm1} + \text{FFN}(\mathbf{X}_{norm1})$$

- **Layer Normalization:** Normalizes features token-by-token using independent parameters γ_2, β_2 :

$$\mathbf{X}_{out} = \text{LayerNorm}_2(\mathbf{X}_{res2})$$

- **Shape:** (B, T, d_{model})
-

Stage 3: Sequential Block Passing

The output tensor $\mathbf{X}_{out}$ from Block 1 becomes the direct input $\mathbf{X}_{in}$ for Block 2, and so on up to Block N . The attention mask is passed unchanged to every block.

Note on Forward Pass Isolation: During this forward pass, raw embeddings E and positional encodings PE are **never re-fetched or re-added**. The residual connections inside each block naturally carry the original positional and semantic signals forward through all N blocks.

Stage 4: Language Modeling Head

After exiting the final block N , the tensor $\mathbf{X}_N \in \mathbb{R}^{B \times T \times d_{model}}$ is converted into token predictions across the vocabulary:

1. Linear Projection (Un-embedding):

$$\mathbf{Z} = \mathbf{X}_N \cdot \mathbf{W}_{LM} + \mathbf{b}_{LM} \quad \text{where } \mathbf{W}_{LM} \in \mathbb{R}^{d_{model} \times V}$$

- **Shape Output:** $\mathbf{Z} \in \mathbb{R}^{B \times T \times V}$ (raw scores/logits for all V vocabulary tokens at every position).

2. Softmax:

$$P(w_i) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

- **Shape Output:** $\mathbb{R}^{B \times T \times V}$ (probability distributions summing to 1.0).

(During training only the N masked positions are sent through the head — see 7-linearization-and-softmax.md, Section 2.)

Stage 5: Forward Pass vs. Training Updates (The Distinction)

Aspect	Forward Pass (Within 1 Sequence Run)	Backward Pass (Across Training Steps / Rounds)
Data Flow	Tensors flow strictly forward from Input $\rightarrow$ Blocks 1 ... $N \rightarrow$ LM Head.	Gradients flow backward from Loss $\rightarrow$ LM Head $\rightarrow$ Blocks $N \dots 1 \rightarrow$ Embedding Matrix E .

Aspect	Forward Pass (Within 1 Sequence Run)	Backward Pass (Across Training Steps / Rounds)
Input Embedding Usage	Embeddings are looked up once at Layer 0. They are not touched or re-added at higher blocks.	The weight values inside matrix E are updated by the optimizer (Adam) using calculated gradients.
State of Weights	Weights ($E, W_Q, W_K, W_V, W_O, \gamma, \beta, W_1, b_1, W_2, b_2, W_{LM}, b_{LM}$) remain constant during computation.	Weights are adjusted after every batch. In training step $k + 1$, the model uses the updated weights for new inputs.
Inference Behavior	Identical to forward pass during training.	Disabled. All weights are frozen and no parameters are updated.

Project-Specific Training and Target Data Preparation Strategy

This project trains the encoder as a **summarizer**, not as a BERT-style masked language model. There is no decoder, so one encoder stack reads the article *and* writes the summary. This setup is called a **prefix-LM** (prefix language model).

Code: `src/data_preprocessing/batching.py` → `CreateTrainingBatch.create_input_tensor` and `build_attention_mask`

1. The data

- **Articles:** about 132,000 pure-Devanagari Nepali news articles, `data/cleaned/{name}.txt`.
- **Targets:** one summary per article, `data/summary/{name}-summary.txt`. The batcher pairs files by `{name}` and skips a summary whose article is missing.
- **One sample = one (article, summary) pair.** Pairs are streamed from disk one at a time, so RAM use stays flat however big the corpus is.

2. Three kinds of "masking", and which ones we use

Kind	What it does	Used here?
Token masking (MLM)	Replace ~15% of input tokens with <code><mask></code> and predict them (Section 2 of 7-linearization-and-softmax.md)	✗ Only in the old pre-training setup. No token is ever replaced with <code><mask></code> here.
Attention masking	Hide some positions from others inside attention (score $\rightarrow -10^9$)	✓ the prefix-LM mask, (B, T, T)
Loss masking	Compute the loss only at some positions	✓ loss only on the summary, (B, T)

When papers or tutorials say “the input should be masked” for summarization, they mean **loss masking**: the model reads the whole article, but is never graded on reproducing it.

3. Step A — Build one sequence per sample

The article and summary are tokenized separately, each with its own budget, then joined:

$\underbrace{\langle s \rangle \text{ article[:766]} \langle /s \rangle}_{\text{prefix: at most 768 tokens}} \quad \underbrace{\text{summary[:255]} \langle /s \rangle}_{\text{target: at most 256 tokens}} \quad \langle \text{pad} \rangle \dots$

- **Budgets, not a fixed split.** $768 + 256 = 1024$ is the *maximum* length (75% / 25%). A short article is **not** padded up to 768. The summary starts right after the article's real $\langle /s \rangle$, and $\langle \text{pad} \rangle$ is only added **at the end** of the row, up to the longest sample in the batch.
- **Truncation keeps the start.** An article longer than 766 tokens loses its right side. News articles put the key facts first, so this keeps the most useful part. The same rule cuts summaries at 255 tokens.
- **How often it happens (2,000 real pairs):** the median article is 1,125 tokens and 66% of articles are truncated. The median summary is 162 tokens and about 6% are truncated.
- **The input row contains the real summary.** This is *teacher forcing*: the true summary tokens go into the model as input, and the attention mask (Step B) stops any position from seeing a token it has to predict.

4. Step B — The attention mask: who may look at whom

`build_attention_mask` builds a (B, T, T) boolean mask (query $\times$ key). A key is visible to a query when it is a real token **and** either it belongs to the prefix **or** it is not after the query:

$$\text{visible}(q, k) = (k < \text{prefix_len} \vee k \leq q) \wedge k < \text{len}$$

Toy example: article a1 a2 a3 and summary s1 s2 (prefix length 5, real length 8, then one $\langle \text{pad} \rangle$):

	keys $\rightarrow$	$\langle s \rangle$	a1	a2	a3	$\langle /s \rangle$	s1	s2	$\langle /s \rangle$	$\langle \text{pad} \rangle$	
query $\langle s \rangle$		✓	✓	✓	✓	✓	•	•	•	•	
a1		✓	✓	✓	✓	✓	•	•	•	•	article rows: the whole article, in both directions (like an encoder); never the summary
a2		✓	✓	✓	✓	✓	•	•	•	•	
a3		✓	✓	✓	✓	✓	•	•	•	•	
$\langle /s \rangle$		✓	✓	✓	✓	✓	•	•	•	•	
s1		✓	✓	✓	✓	✓	✓	•	•	•	summary rows: the whole article + earlier summary tokens only (causal, like a decoder)
s2		✓	✓	✓	✓	✓	✓	✓	•	•	
$\langle /s \rangle$		✓	✓	✓	✓	✓	✓	✓	✓	•	

Compared with an encoder-decoder Transformer, the top-left block does the encoder's job, the bottom-left block does cross-attention's job, and the bottom-right triangle is the decoder's causal mask. $\langle \text{pad} \rangle$ keys are hidden from everyone.

5. Step C — Targets and the loss mask

Position p is trained to predict the token at $p + 1$. The loss mask keeps only the positions whose next token is a summary token, from the prefix's closing $\langle /s \rangle$ to the last summary token:

$$\text{loss_mask}[p] = (p \geq \text{prefix_len} - 1) \wedge (p < \text{len} - 1)$$

position:	0	1	2	3	4	5	6	7	8
input:	<s>	a1	a2	a3	</s>	s1	s2	</s>	<pad>
next token:	a1	a2	a3	</s>	s1	s2	</s>	<pad>	–
loss?	x	x	x	x	✓	✓	✓	x	x

- **The article positions get no loss.** This is the “input is masked” part.
- **targets has shape (N) .** The code gathers only the next tokens at the N loss positions (here [s1, s2, </s>]), instead of keeping a (B, T) label row filled with “ignore” values.
- **The final </s> is a target.** The model learns when to stop, which is how inference knows the summary is finished.
- The LM head runs only on those N positions: $(N, 512) \rightarrow (N, V)$. The gradient is scattered back into $(B, T, 512)$ in backward (9-backpropagation.md, Step 2).

Common misconception: “the input holds 768 article tokens and then a mask, and the target holds 768 masked positions and then 255 summary tokens.” That is only true when the article is exactly 766 tokens long. In general, the input holds the **real** article (whatever its length, up to 766) followed by the **real** summary, with nothing masked out. The article/summary boundary sits at `prefix_len`, which is different for every sample. Masking happens in the attention mask and the loss mask, never in the token IDs.

6. Worked example: a real batch with $B = 2$

	100001.txt	100089.txt
Article tokens	2,046 $\rightarrow$ cut to 766	427 (fits)
Prefix <s> article </s>	768	429
Summary tokens (+ </s>)	213 + 1 = 214	123 + 1 = 124
Real length	982	553
<pad> added	0	429
Loss positions	767 ... 980 $\rightarrow$ 214	428 ... 551 $\rightarrow$ 124

The batch has $T = 982$ (the longest row, not 1024), so `token_ids` is $(2, 982)$, the attention mask is $(2, 982, 982)$ and the loss mask is $(2, 982)$. It has $N = 214 + 124 = 338$ loss positions, so `targets` is (338) and the LM head produces $(338, V)$ probabilities.

7. Inference uses the same layout

`src/inference/greedy_decoder.py` $\rightarrow$ `greedy_summarize` starts from `<s> article[:766] </s>` with no summary. It builds the same prefix-LM mask, runs the LM head on the last position only, appends the argmax token, and repeats until `</s>` or 256 summary tokens. The model sees exactly the pattern it was trained on, except that the summary tokens are its own predictions instead of the ground truth.

One Training Step in This Project

1. `batcher.create_input_tensor()` $\rightarrow$ token IDs (B, T) , attention mask (B, T, T) , loss mask (B, T) , targets (N)
2. `model.forward(..., output_mask)` $\rightarrow$ probabilities at the N summary positions (N, V)
3. `cross_entropy_loss(...)` $\rightarrow$ loss (a number) and `d_logits` (N, V)

4. `model.backward(d_logits)` → a gradient for every weight and bias
 5. `optimizer.step(params, grads)` → Adam updates every weight in place
 (src/training/train.py → train_step. Start training with `python -m scripts.train_model.`)

Total Parameter Count

Component	Count
Input embedding E (30000×512)	15,360,000
6 encoder blocks $\times$ 3,150,336	18,902,016
LM head ($W_{LM} + b_{LM}$)	15,390,000
Positional encoding	0 (fixed, not learned)
Total	49,652,016

Chapter 10

Backpropagation: How Every Weight and Bias Gets Updated

Documents 1-8 describe the **forward pass**: token IDs go in, probabilities come out. This document describes the **backward pass**: how the loss tells every single weight and bias in the encoder which way to move, and by how much.

In one sentence: starting from the loss, we walk the forward pass in reverse, and at every layer we use the chain rule to compute (a) the gradient for that layer's own parameters and (b) the gradient to hand to the layer below.

Code: every layer has a `backward()` method next to its `forward()` (`src/models/layers/*.py`); `TransformerEncoder.backward` in `src/models/encoder.py` calls them in reverse order; `src/training/loss.py` and `src/training/optimizer.py` hold the loss and Adam.

0. The Big Picture

Notation

For any tensor **A** in the network, we write

$$d\mathbf{A} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}}$$

"how much the loss $\mathcal{L}$ changes when each number in **A** is nudged up a little". **$d\mathbf{A}$ always has exactly the same shape as **A**.** That single rule catches most mistakes: dW_1 must be (512, 2048) because W_1 is (512, 2048).

- A **positive** gradient means "increasing this number increases the loss" → move it **down**.
- A **negative** gradient means "increasing this number decreases the loss" → move it **up**.

Forward saves, backward uses

During the forward pass every layer stores (“caches”) the inputs it will need later — e.g. the FFN keeps its input $\mathbf{x}$ and its ReLU output. The backward pass reads those caches. This is why training uses much more memory than inference.

The order

FORWARD: `ids → Embedding → +PE → [Block 1] → ... → [Block 6] → LM head → softmax → loss`

BACKWARD: `loss → softmax+CE → LM head → [Block 6] → ... → [Block 1] → PE → Embedding`

|
inside each block (reverse): ▼
Add&Norm 2 → FFN → Add&Norm 1 → Multi-Head Attention

Every learnable parameter in the encoder

#	Parameter	Where	Shape	Count
1	E	Input embedding	(30000, 512)	15,360,000
2-5	W_Q, W_K, W_V, W_O	Attention (×6 blocks)	(512, 512) each	262,144 each
6-7	γ_1, β_1	Add & Norm 1 (×6)	(512) each	512 each
8-9	W_1, b_1	FFN (×6)	(512, 2048), (2048)	1,048,576 + 2,048
10-11	W_2, b_2	FFN (×6)	(2048, 512), (512)	1,048,576 + 512
12-13	γ_2, β_2	Add & Norm 2 (×6)	(512) each	512 each
14-15	W_{LM}, b_{LM}	LM head	(512, 30000), (30000)	15,360,000 + 30,000
Total				49,652,016

The positional encoding PE is fixed, so it has **no** gradient to compute. Each of the 6 blocks has its own copy of parameters 2-13, and each copy gets its own gradient.

1. Three Rules That Cover Everything

Rule 1 — The chain rule

If $\mathbf{y} = f(\mathbf{x})$ and the loss depends on $\mathbf{y}$, then

$$d\mathbf{x} = d\mathbf{y} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$$

Each layer only needs to know its **own** local derivative $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$; the incoming $d\mathbf{y}$ already contains everything that happens above it.

Rule 2 — The linear layer (used 9 times per block + the LM head)

Almost every weight in the Transformer sits in a layer of the form

$$\mathbf{Y} = \mathbf{XW} + b \quad \mathbf{X} : (\dots, n_{in}), \mathbf{W} : (n_{in}, n_{out}), b : (n_{out}), \mathbf{Y} : (\dots, n_{out})$$

Given the incoming gradient $d\mathbf{Y}$:

$$d\mathbf{W} = \mathbf{X}^T d\mathbf{Y} \quad db = \sum_{\text{rows}} d\mathbf{Y} \quad d\mathbf{X} = d\mathbf{Y} \mathbf{W}^T$$

- **Why $\mathbf{X}^T d\mathbf{Y}$?** Weight W_{jk} connects input feature j to output feature k . Its effect on the loss is $(\text{input } j) \times (\text{gradient at output } k)$, added up over every token that used it.
- **Why sum for db ?** The same bias is added to every token, so its gradient is the sum of all tokens' output gradients.
- **Why $d\mathbf{Y} \mathbf{W}^T$?** Each input feature fed into every output through $\mathbf{W}$; we send the gradient back along the same connections.

The weights are shared by every token in every sentence, so for 3D tensors (B, T, n) we flatten to $(B \cdot T, n)$ first: the weight gradient is summed over all $B \times T$ token positions. (In code: `matmul_weight_grad` in `src/models/layers/functional.py`.)

Example — 2 tokens, $n_{in} = n_{out} = 2$:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 0.5 & -1 \\ 2 & 0 \end{bmatrix}, \quad b = [0.1, 0] \Rightarrow \mathbf{Y} = \begin{bmatrix} 4.6 & -1 \\ -0.4 & -3 \end{bmatrix}$$

Suppose the layer above sends back $d\mathbf{Y} = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}$. Then

$$d\mathbf{W} = \mathbf{X}^T d\mathbf{Y} = \begin{bmatrix} 1 & 3 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} -2 & 6 \\ 3 & -2 \end{bmatrix}$$

$$db = [1 + (-1), 0 + 2] = [0, 2] \quad d\mathbf{X} = d\mathbf{Y} \mathbf{W}^T = \begin{bmatrix} 0.5 & 2 \\ -2.5 & -2 \end{bmatrix}$$

Rule 3 — Addition copies, re-use adds up

- **Addition $\mathbf{c} = \mathbf{a} + \mathbf{b}$:** $\frac{\partial \mathbf{c}}{\partial \mathbf{a}} = \frac{\partial \mathbf{c}}{\partial \mathbf{b}} = 1$, so both inputs receive the **same** gradient: $d\mathbf{a} = d\mathbf{b} = d\mathbf{c}$. This is why residual connections are a "gradient highway" — nothing shrinks the gradient on the skip path.
- **Re-use:** if a tensor is used in two places (e.g. $\mathbf{X}_{in}$ goes into attention **and** into the residual), its gradient is the **sum** of the gradients coming back from both places.

2. Step 1 — Loss and Softmax: $d\mathbf{Z}$

Forward (for the N masked positions, see `7-linearization-and-softmax.md`):

$$\mathbf{P} = \text{softmax}(\mathbf{Z}), \quad \mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \log P_{n, \text{target}_n}$$

Differentiating softmax and cross-entropy **together** gives a famously simple result:

$$d\mathbf{Z} = \frac{1}{N} (\mathbf{P} - \mathbf{Y}_{\text{one-hot}}) \quad \text{shape } (N, V)$$

i.e. "predicted probability minus the truth": subtract 1 at the correct token, leave the others as they are, divide by N .

Why it simplifies (for one position)

$\mathcal{L} = -\log P_c = -z_c + \log \sum_j e^{z_j}$ where c is the correct token. $\frac{\partial \mathcal{L}}{\partial z_i} = -[i = c] + \frac{e^{z_i}}{\sum_j e^{z_j}} = P_i - [i = c]$. **Example** (the toy vocabulary from document 7, correct token = "नेपाल", $N = 1$):

token	P	one-hot $\mathbf{Y}$	$d\mathbf{Z} = \mathbf{P} - \mathbf{Y}$
"नेपाल" ✓	0.6381	1	-0.3619 → push this logit up
"सुन्दर"	0.2347	0	0.2347 → push down
"देश"	0.0954	0	0.0954 → push down
"हो"	0.0318	0	0.0318 → push down

The gradients always sum to 0: probability taken from wrong tokens is given to the right one. Wrong tokens that got more probability get pushed down harder.

3. Step 2 — LM Head: W_{LM}, b_{LM}

Forward: $\mathbf{Z} = \mathbf{X}_N W_{LM} + b_{LM}$ with $\mathbf{X}_N$ the $(N, 512)$ encoder vectors at the masked positions. This is Rule 2:

$$dW_{LM} = \mathbf{X}_N^T d\mathbf{Z} \quad (512, V) \quad db_{LM} = \sum_n d\mathbf{Z}_n \quad (V) \quad d\mathbf{X}_N = d\mathbf{Z} W_{LM}^T \quad (N, 512)$$

Example ($d_{model} = 2, V = 4$, one position): $\mathbf{x} = [1, -2]$, $d\mathbf{z}$ from Step 1, and $W_{LM} = \begin{bmatrix} 0.1 & 0.2 & 0 & -0.1 \\ 0.3 & -0.2 & 0.1 & 0 \end{bmatrix}$.

$$dW_{LM} = \mathbf{x}^T d\mathbf{z} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} [-0.3619, 0.2347, 0.0954, 0.0318] = \begin{bmatrix} -0.3619 & 0.2347 & 0.0954 & 0.0318 \\ 0.7239 & -0.4695 & -0.1909 & -0.0635 \end{bmatrix}$$

$$db_{LM} = [-0.3619, 0.2347, 0.0954, 0.0318] \quad d\mathbf{x} = d\mathbf{z} W_{LM}^T = [0.0076, -0.1460]$$

Back to $(B, T, 512)$: only the N masked positions went through the head, so the code creates a zero tensor of shape $(B, T, 512)$ and writes the N rows of $d\mathbf{X}_N$ into their original positions. Unmasked positions start with gradient 0 — but they will still receive gradient later through attention, because the masked tokens attended to them.

Weight tying (if enabled): $W_{LM} = E^T$, so dW_{LM}^T is added to dE (Step 8) instead of updating a separate matrix.

4. Step 3 — Second Add & Norm: γ_2, β_2

Forward: $\mathbf{X}_{out} = \text{LayerNorm}_2(\mathbf{X}_{norm1} + \mathbf{X}_{ffn_out})$. We go through it in reverse: first LayerNorm, then the addition.

LayerNorm backward

Forward, for each token vector $\mathbf{x}$ (length $D = 512$):

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta$$

Parameter gradients (summed over all $B \times T$ tokens, since γ, β are shared):

$$d\gamma = \sum_{b,t} d\mathbf{y} \odot \hat{\mathbf{x}} \quad d\beta = \sum_{b,t} d\mathbf{y} \quad \text{both shape (512)}$$

(Same idea as Rule 2: γ_j multiplies $\hat{x}_j$, and β_j is a bias.)

Input gradient. Let $d\hat{\mathbf{x}} = d\mathbf{y} \odot \gamma$. Because μ and σ^2 are computed from all D numbers of the same token, every input affects every output, and the result is:

$$d\mathbf{x} = \frac{1}{\sqrt{\sigma^2 + \epsilon}} (d\hat{\mathbf{x}} - \text{mean}(d\hat{\mathbf{x}}) - \hat{\mathbf{x}} \odot \text{mean}(d\hat{\mathbf{x}} \odot \hat{\mathbf{x}}))$$

Intuition for the two subtracted terms: normalization throws away the vector's **average** and its **overall scale**. So any part of the gradient that would only shift the average (first term) or only stretch the scale (second term) has no effect after normalization and is removed. A consequence you can check: the entries of $d\mathbf{x}$ always sum to 0.

Example — the token from 4-add_and_normalization.md: $\mathbf{x} = [1.5, 1.0, 4.0, 5.5]$, $\sqrt{\sigma^2 + \epsilon} = 1.8371$, $\hat{\mathbf{x}} = [-0.8165, -1.0887, 0.5443, 1.3608]$, $\gamma = 1$, $\beta = 0$. Incoming gradient $d\mathbf{y} = [1, 0, 0, -1]$.

1. $d\gamma = d\mathbf{y} \odot \hat{\mathbf{x}} = [-0.8165, 0, 0, -1.3608]$
 2. $d\beta = d\mathbf{y} = [1, 0, 0, -1]$
 3. $d\hat{\mathbf{x}} = d\mathbf{y} \odot \gamma = [1, 0, 0, -1]$; $\text{mean}(d\hat{\mathbf{x}}) = 0$
 4. $d\hat{\mathbf{x}} \odot \hat{\mathbf{x}} = [-0.8165, 0, 0, -1.3608]$; $\text{mean} = -0.5443$
 5. $d\mathbf{x} = \frac{1}{1.8371} ([1, 0, 0, -1] - 0 + 0.5443 \cdot \hat{\mathbf{x}}) = \frac{1}{1.8371} [0.5556, -0.5926, 0.2963, -0.2593]$
- $d\mathbf{x} = [0.3024, -0.3226, 0.1613, -0.1411]$ (sum = 0 ✓)

The addition

$\mathbf{X}_{res2} = \mathbf{X}_{norm1} + \mathbf{X}_{ffn_out}$, so by Rule 3 **both** $\mathbf{X}_{norm1}$ (skip path) and $\mathbf{X}_{ffn_out}$ (FFN path) receive the same gradient $d\mathbf{X}_{res2}$.

5. Step 4 — Feed-Forward Network: W_2, b_2, W_1, b_1

Forward: $\mathbf{h} = \mathbf{x}W_1 + b_1$, $\mathbf{a} = \text{ReLU}(\mathbf{h})$, $\mathbf{y} = \mathbf{a}W_2 + b_2$. Backward, top to bottom:

Step	Formula	Shape
Linear 2 (Rule 2)	$dW_2 = \mathbf{a}^T d\mathbf{y}$, $db_2 = \sum d\mathbf{y}$, $d\mathbf{a} = d\mathbf{y}$	(2048, 512), (512), ($B, T, 2048$)
ReLU	$d\mathbf{h} = d\mathbf{a} \odot [\mathbf{h} > 0]$	($B, T, 2048$)
Linear 1 (Rule 2)	$dW_1 = \mathbf{x}^T d\mathbf{h}$, $db_1 = \sum d\mathbf{h}$, $d\mathbf{x} = d\mathbf{h}$	(512, 2048), (2048), ($B, T, 512$)

ReLU's derivative is 1 where the input was positive and 0 elsewhere: units that were switched off in the forward pass pass **no** gradient back and their incoming weights are not updated for that token.

Example — the FFN from 5-feed_forward.md ($\mathbf{x} = [1, -1]$, $\mathbf{h} = [1, -2, 1, 0.5]$, $\mathbf{a} = [1, 0, 1, 0.5]$), with incoming gradient $d\mathbf{y} = [1, -1]$:

1. $dW_2 = \mathbf{a}^T d\mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0.5 \end{bmatrix} [1, -1] = \begin{bmatrix} 1 & -1 \\ 0 & 0 \\ 1 & -1 \\ 0.5 & -0.5 \end{bmatrix}$, $db_2 = [1, -1]$
 2. $d\mathbf{a} = d\mathbf{y} W_2^T = [1, -1, 0, -3]$
 3. ReLU mask $[\mathbf{h} > 0] = [1, 0, 1, 1] \rightarrow d\mathbf{h} = [1, 0, 0, -3]$ (unit 2 was off, so its gradient is blocked)
 4. $dW_1 = \mathbf{x}^T d\mathbf{h} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} [1, 0, 0, -3] = \begin{bmatrix} 1 & 0 & 0 & -3 \\ -1 & 0 & 0 & 3 \end{bmatrix}$, $db_1 = [1, 0, 0, -3]$
 5. $d\mathbf{x} = d\mathbf{h} W_1^T = [1 \cdot 1 + (-3) \cdot 2, 1 \cdot 0 + (-3) \cdot 1] = [-5, -3]$
-

6. Step 5 — First Add & Norm: γ_1, β_1

$\mathbf{X}_{norm1}$ was used **twice** in the forward pass: as the FFN input and on the skip path into Add & Norm 2. By Rule 3 its total gradient is the sum:

$$d\mathbf{X}_{norm1} = \underbrace{d\mathbf{X}_{res2}}_{\text{skip path}} + \underbrace{d\mathbf{x}_{FFN}}_{\text{through the FFN}}$$

Then exactly the same LayerNorm backward as Step 3 gives $d\gamma_1, d\beta_1$ and $d\mathbf{X}_{res1}$, and the addition $\mathbf{X}_{res1} = \mathbf{X}_{in} + \mathbf{X}_{attn}$ copies $d\mathbf{X}_{res1}$ to both $\mathbf{X}_{in}$ (skip) and $\mathbf{X}_{attn}$ (attention output).

7. Step 6 — Multi-Head Attention: W_O, W_Q, W_K, W_V

Forward recap (per head, with $\mathbf{P}$ the attention weights):

$$\mathbf{Q} = \mathbf{X} W_Q, \mathbf{K} = \mathbf{X} W_K, \mathbf{V} = \mathbf{X} W_V, \mathbf{S} = \frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}, \mathbf{P} = \text{softmax}(\mathbf{S}), \mathbf{A} = \mathbf{P} \mathbf{V}, \text{out} = \text{Concat}(\mathbf{A}) W_O$$

Backward, one line per forward operation, in reverse:

#	Undo	Gradient	Shape
a	output projection (Rule 2)	$dW_O = \text{Concat}^T d\text{out}$, $d\text{Concat} = d\text{out} W_O^T$	$(512, 512)$, $(B, T, 512)$
b	concatenation	split $d\text{Concat}$ back into heads $\rightarrow d\mathbf{A}$	(B, h, T, d_k)
c	$\mathbf{A} = \mathbf{P} \mathbf{V}$	$d\mathbf{P} = d\mathbf{A} \mathbf{V}^T$, $d\mathbf{V} = \mathbf{P}^T d\mathbf{A}$	(B, h, T, T) , (B, h, T, d_k)
d	softmax (per row)	$d\mathbf{S} = \mathbf{P} \odot (d\mathbf{P} - \sum_j (d\mathbf{P} \odot \mathbf{P})_j)$	(B, h, T, T)
e	scaling by $\frac{1}{\sqrt{d_k}}$	$d\mathbf{S} \leftarrow d\mathbf{S} / \sqrt{d_k}$	(B, h, T, T)
f	$\mathbf{Q} \mathbf{K}^T$	$d\mathbf{Q} = d\mathbf{S} \mathbf{K}$, $d\mathbf{K} = d\mathbf{S}^T \mathbf{Q}$	(B, h, T, d_k) each
g	split into heads	merge $d\mathbf{Q}, d\mathbf{K}, d\mathbf{V}$ back to $(B, T, 512)$	$(B, T, 512)$

#	Undo	Gradient	Shape
h	input projections (Rule 2)	$dW_Q = \mathbf{X}^T d\mathbf{Q}$, $dW_K = \mathbf{X}^T d\mathbf{K}$, $dW_V = \mathbf{X}^T d\mathbf{V}$	(512, 512) each
i	$\mathbf{X}$ used 3 times (Rule 3)	$d\mathbf{X} = d\mathbf{Q}W_Q^T + d\mathbf{K}W_K^T + d\mathbf{V}W_V^T$	(B, T, 512)

The mask needs no special backward code. Masked positions have $P = 0$, and row (d) multiplies by $\mathbf{P}$, so their $d\mathbf{S}$ is exactly 0: [PAD] keys receive no gradient.

Softmax backward, tiny example: $\mathbf{P} = [0.5, 0.5, 0]$, $d\mathbf{P} = [1, 0, 2]$. Then $\sum_j P_j dP_j = 0.5$ and

$$d\mathbf{S} = [0.5 \cdot (1 - 0.5), 0.5 \cdot (0 - 0.5), 0 \cdot (2 - 0.5)] = [0.25, -0.25, 0]$$

Full example — the attention head from 3-multi-head-attention.md ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and weights $\mathbf{P}$ as computed there; token 3 is [PAD]). Suppose only token 1's output gets a gradient,

$$d\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}:$$

1. $d\mathbf{V} = \mathbf{P}^T d\mathbf{A}$: token 1 used 50% of $\mathbf{V}_1$ and 50% of $\mathbf{V}_2 \rightarrow d\mathbf{V} = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \\ 0 & 0 \end{bmatrix}$ (the [PAD] Value gets nothing)
2. $d\mathbf{P}$ (row 1) $= [1, 0] \cdot \mathbf{V}^T = [1, 3, 9]$
3. Softmax backward: $\sum_j P_j dP_j = 0.5 \cdot 1 + 0.5 \cdot 3 + 0 \cdot 9 = 2 \rightarrow d\mathbf{S}$ (row 1) $= [0.5(1 - 2), 0.5(3 - 2), 0 \cdot (9 - 2)] = [-0.5, 0.5, 0]$
4. Divide by $\sqrt{2}$: $[-0.3536, 0.3536, 0]$
5. $d\mathbf{Q}$ (row 1) $= -0.3536 \cdot \mathbf{K}_1 + 0.3536 \cdot \mathbf{K}_2 = -0.3536 \cdot [1, 0] + 0.3536 \cdot [1, 1] = [0, 0.3536]$
6. $d\mathbf{K} = d\mathbf{S}^T \mathbf{Q}$: $d\mathbf{K}_1 = [-0.3536, 0]$, $d\mathbf{K}_2 = [0.3536, 0]$, $d\mathbf{K}_3 = [0, 0]$

Reading it: $d\mathbf{A} = [1, 0]$ means "the loss goes up when the first number of token 1's output goes up". Key 2's Value has a larger first number (3) than key 1's (1), so gradient descent — which moves *against* $d\mathbf{S} = [-0.35, +0.35]$ — raises the score for key 1 and lowers it for key 2, shifting token 1's attention toward the smaller Value. Even though the [PAD] Value [9, 9] has the largest dP , it gets **zero** gradient because of the mask.

8. Step 7 — Through the Stack of Blocks

At the bottom of each block, $\mathbf{X}_{in}$ was used twice (attention input and skip path), so:

$$d\mathbf{X}_{in} = \underbrace{d\mathbf{X}_{res1}}_{\text{skip path}} + \underbrace{d\mathbf{X}_{\text{attention}}}_{\text{Step 6 (i)}}$$

This $d\mathbf{X}_{in}$ of Block k is the $d\mathbf{X}_{out}$ of Block $k - 1$. The code simply loops over the blocks in reverse:

```
for block in reversed(self.blocks):
    d_hidden = block.backward(d_hidden)
```

Thanks to the skip paths, part of the gradient reaches Block 1 having passed through only additions and LayerNorms — never through a long chain of weight matrices. This is why

the Transformer's early layers and embeddings keep learning even when the network is deep.

9. Step 8 — Positional Encoding and Input Embedding: E

Positional encoding: $\mathbf{X}_0 = \mathbf{X}_{emb} + PE$. PE is a constant, so by Rule 3 the gradient passes through unchanged: $d\mathbf{X}_{emb} = d\mathbf{X}_0$. Nothing to update.

Embedding: forward was $\mathbf{X}_{emb}[b, t] = E[\text{id}_{b,t}] \cdot \sqrt{d_{model}}$ — just picking rows and scaling. So each position sends its gradient (times $\sqrt{d_{model}}$) back to **the row it came from**:

$$dE[r] = \sqrt{d_{model}} \sum_{(b,t): \text{id}_{b,t}=r} d\mathbf{X}_0[b, t]$$

- Tokens **not** in the batch: gradient row is all zeros.
- A token that appears **several times**: its gradients are **added** (the code uses `np.add.at`, which accumulates repeats correctly).

Example ($d_{model} = 4$, so $\sqrt{4} = 2$; vocabulary of 6; sentence IDs $[2, 5, 2]$ — token 2 appears twice):

position	ID	$d\mathbf{X}_0$
0	2	$[0.1, 0, -0.2, 0.3]$
1	5	$[0, 0.5, 0, 0]$
2	2	$[0.2, 0.1, 0, -0.1]$

$$dE[2] = 2 \times ([0.1, 0, -0.2, 0.3] + [0.2, 0.1, 0, -0.1]) = [0.6, 0.2, -0.4, 0.4]$$

$$dE[5] = 2 \times [0, 0.5, 0, 0] = [0, 1, 0, 0]$$

$$dE[0] = dE[1] = dE[3] = dE[4] = [0, 0, 0, 0]$$

Weight tying: if $W_{LM} = E^T$, then add dW_{LM}^T (from Step 2) to dE — the same matrix gets gradient from both ends of the network.

10. Updating the Parameters

After `model.backward()` every parameter W has a gradient dW of the same shape.

Plain gradient descent (the idea)

$$W \leftarrow W - \eta \cdot dW \quad (\eta = \text{learning rate, e.g. } 10^{-4})$$

Adam (what this project uses)

Plain gradient descent uses one step size for every weight. **Adam** keeps two running averages **for each individual number** in every parameter tensor (so m and ν have the same shape as W):

$$m \leftarrow \beta_1 m + (1 - \beta_1) dW \quad (\text{average gradient — “momentum”, } \beta_1 = 0.9)$$

$$\nu \leftarrow \beta_2 \nu + (1 - \beta_2) dW^2 \quad (\text{average squared gradient, } \beta_2 = 0.999)$$

Because m and ν start at 0, they are too small in the first steps; **bias correction** fixes that (t = step number):

$$\hat{m} = \frac{m}{1 - \beta_1^t} \quad \hat{\nu} = \frac{\nu}{1 - \beta_2^t}$$

$$W \leftarrow W - \eta \cdot \frac{\hat{m}}{\sqrt{\hat{\nu}} + \epsilon} \quad (\epsilon = 10^{-8})$$

$\hat{m}/\sqrt{\hat{\nu}}$ is roughly “average direction $\div$ typical size”, so each weight moves by about η per step regardless of whether its raw gradients are tiny or huge. Weights whose gradient keeps flipping sign get smaller steps.

Example — one weight $w = 0.5$, $\eta = 0.001$:

step t	gradient dw	m	ν	$\hat{m}$	$\hat{\nu}$	step = $\eta \hat{m}/\sqrt{\hat{\nu}}$	new w
1	0.2	0.02	0.00004	0.2	0.04	0.001	0.499
2	-0.1	0.008	0.00004996	0.0421050	0.0249920	0.000266	0.498734

Step 1: $m = 0.1 \cdot 0.2 = 0.02$, $\nu = 0.001 \cdot 0.04 = 0.00004$; corrected $\hat{m} = 0.02/0.1 = 0.2$, $\hat{\nu} = 0.00004/0.001 = 0.04$; step = $0.001 \cdot 0.2/0.2 = 0.001$. Step 2: the gradient flipped sign, so momentum partly cancels and the step shrinks to 0.000266 — Adam is cautious when the direction is uncertain.

Gradient clipping (safety belt)

Before the Adam update, the code measures the size of **all** gradients together (global norm):

$$\|g\| = \sqrt{\sum_{\text{all parameters}} \sum_{\text{all entries}} dW^2}$$

If $\|g\| > 1.0$ (max_grad_norm), every gradient is multiplied by $1.0/\|g\|$. Example: two gradients [3] and [4] have norm $\sqrt{9 + 16} = 5$, so they become [0.6] and [0.8] — same direction, safe size. This prevents one bad batch from wrecking the weights.

11. Summary: Every Gradient in One Table

Parameter	Gradient formula	Code
W_{LM}, b_{LM}	$\mathbf{X}_N^T d\mathbf{Z}, \sum d\mathbf{Z}$, with $d\mathbf{Z} = (\mathbf{P} - \mathbf{Y})/N$	LanguageModelingHead.backward
γ_2, β_2	$\sum d\mathbf{y} \odot \hat{\mathbf{x}}, \sum d\mathbf{y}$	LayerNormalization.backward
W_2, b_2	$\mathbf{a}^T d\mathbf{y}, \sum d\mathbf{y}$	PositionwiseFeedForward.backward
W_1, b_1	$\mathbf{x}^T d\mathbf{h}, \sum d\mathbf{h}$, with $d\mathbf{h} = d\mathbf{a} \odot [\mathbf{h} > 0]$	PositionwiseFeedForward.backward
γ_1, β_1	$\sum d\mathbf{y} \odot \hat{\mathbf{x}}, \sum d\mathbf{y}$	LayerNormalization.backward
W_O	$\text{Concat}^T d\text{out}$	MultiHeadAttention.backward
W_Q, W_K, W_V	$\mathbf{X}^T d\mathbf{Q}, \mathbf{X}^T d\mathbf{K}, \mathbf{X}^T d\mathbf{V}$	MultiHeadAttention.backward
E	$\sqrt{d_{model}} \cdot (\text{sum of } d\mathbf{X}_0 \text{ rows per token ID})$	InputEmbedding.backward
PE	— (fixed, no gradient)	PositionalEncoding.backward

Sums ($\sum$) run over every token position in the batch, because the same parameter is used for every token.

12. How We Know the Formulas Are Right: Gradient Checking

Every formula above can be tested numerically. Pick one weight w , nudge it by a tiny h (e.g. 10^{-5}), and measure the loss on both sides:

$$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\mathcal{L}(w + h) - \mathcal{L}(w - h)}{2h}$$

If the backward pass is correct, this number matches the computed dw . This check was run on a tiny version of this encoder (every parameter tensor, with and without weight tying); the largest relative difference was about 2×10^{-7} , and every worked example in this document was verified the same way.